МАТЕРИАЛЫ ДЛЯ ГОСУДАРСТВЕННОЙ РЕГИСТРАЦИИ ПРОГРАММЫ ДЛЯ ЭВМ
АРМ
МОДУЛЬ «ЭКОЛОГИЯ»
ОПИСАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ, ФРАГМЕНТЫ ИСХОДНОГО ТЕКСТА И ЭКРАННЫЕ ФОРМЫ

Адрес модуля: http://msk-arm-gs01.ntcees.ru/ecology/

Дата подготовки материалов: 29 октября 2026 года

Состав: 30 экранных представлений модуля «Экология», 4 скрытые совместимые вкладки, отчетные данные ПГ и ЗВ, анализ, расчеты действующих и новых ТЭС, сводные расчеты, агрегированные суммы, тексты формул и рассмотрение топливных пакетов.

Документ подготовлен как отдельный файл, предусмотренный пунктом «Исходный текст программы (листинг)» шаблона «Сведения для регистрации программы». Приведены фрагменты, а не полный исходный текст программы.

Основание описания: текущие исходные файлы рабочего проекта и экранные формы действующего сайта. Номер релиза серверного приложения отдельно не устанавливался: совпадение серверной сборки со всеми локальными исходными файлами не утверждается.

СОДЕРЖАНИЕ

1. Сведения по форме предоставленного шаблона 4
2. Модуль «Экология»: назначение и архитектура 6
   2.1. Страницы, вкладки и границы описания 6
   2.2. Программные слои и модели данных 6
   2.3. Права, версия БД и журнал 8
   2.4. Условия получения экранных форм 9
3. Главная страница и отчетные данные ГИС ТЭК 11
   3.1. Главная страница модуля 11
   3.2. Парниковые газы: отчетные данные ГИС ТЭС 12
   3.3. Загрязняющие вещества: отчетные данные ГИС ТЭС 16
4. Анализ отчетных данных и сопоставление с модулем «Топливо» 20
   4.1. Парниковые газы: анализ с котельными 20
   4.2. Парниковые газы: анализ без котельных 22
   4.3. Загрязняющие вещества: анализ с котельными 25
   4.4. Загрязняющие вещества: анализ без котельных 28
5. Расчет по действующим ТЭС 32
   5.1. Данные по модулю Топливо 32
   5.2. Характеристики топлив и продуктов сгорания 34
   5.3. Коэффициенты очистки и подавления 37
   5.4. Зола: выбросы твердых веществ 39
   5.5. Сера: выбросы диоксида серы 41
   5.6. Азот: выбросы оксидов азота 43
   5.7. CO: выбросы монооксида углерода 45
   5.8. ПГ: прямой расчет CO2 47
   5.9. ПГ через ТДж: расчет CO2 по энергии топлива 49
   5.10. Объемы и удельные выбросы действующих ТЭС 51
6. Расчет по новым ТЭС 54
   6.1. Принятые топливные показатели новых ТЭС 54
   6.2. Нормативы удельных выбросов новых ТЭС 56
   6.3. Объемы выбросов новых ТЭС 59
7. Сводные данные расчетов 61
   7.1. Общие топливные показатели действующих и новых ТЭС 61
   7.2. Объемы выбросов по России 62
   7.3. Удельные выбросы по России 64
   7.4. Динамика изменения выбросов 66
8. Агрегированные суммы отчетно-аналитического контура 69
   8.1. ПГ: агрегированные суммы с котельными 69
   8.2. ПГ: агрегированные суммы без котельных 71
   8.3. ЗВ: агрегированные суммы с котельными 74
   8.4. ЗВ: агрегированные суммы без котельных 76
9. Тексты подсказок формул 80
   9.1. Редактор текстов подсказок формул 80
10. Рассмотрение данных, переданных из модуля «Топливо» 83
   10.1. Рассмотрение пакета данных из модуля «Топливо» 83
11. Сквозная логика и взаимодействие с другими модулями 85
   11.1. Отчет, выверенные параметры и агрегаты 85
   11.2. Передача топлива и расчет выбросов ТЭС 86

11.3. Формулы, единицы измерения и отсутствующие данные 88
11.4. Обмен Excel и интерактивные действия 89
12. Полнота описания модуля «Экология» и источники 92
12.1. Перечень экранных представлений 92
12.2. Скрытые совместимые вкладки сводного расчета 93
12.3. Маршруты страниц и служебные операции 95
12.4. Проверка и ограничения 96

1. Сведения по форме предоставленного шаблона

№	Сведения по шаблону	Содержание для модуля «Экология»
1	Название программы (полное и сокращенное)	Программа: АРМ (обозначение, предоставленное заявителем). Полное регистрационное наименование подлежит уточнению. Описываемая часть: модуль «Экология» (ecology).
2	Назначение, область применения, функциональные возможности (в реферативной форме); ~ 10 строк.	Модуль «Экология» программы АРМ хранит и отображает отчетные данные ГИС ТЭК о парниковых газах и загрязняющих веществах. Поддерживает импорт и экспорт Excel, выверку параметров анализа и сопоставление с топливными и генерационными показателями. Рассчитывает выбросы действующих и новых ТЭС на основе принятых топливных данных, характеристик, коэффициентов и нормативов. Формирует сводные расчетные таблицы и агрегированные суммы отчетно-аналитического контура. Обеспечивает рассмотрение пакетов, переданных из модуля «Топливо», и редактирование текстов подсказок формул. Работает через веб-интерфейс с разграничением прав и учетом выбранной версии БД.
3	Тип ЭВМ	Клиент-серверная веб-система. Сервер и клиентские персональные компьютеры; работа пользователя через веб-браузер. Точная аппаратная конфигурация заявляемого экземпляра подлежит уточнению.
4	Язык программирования	Python; JavaScript; шаблоны HTML/Jinja2 и CSS. Доступ к реляционным данным через SQLAlchemy (СУБД PostgreSQL).
5	Тип и версия используемой ОС	В документации проекта описано серверное развертывание на Astra Linux 1.8.4. Текущий рабочий каталог находится в среде Windows. ОС и версия заявляемого экземпляра подтверждаются заявителем.
6	Объем программы для ЭВМ в машиночитаемой форме в единицах, кратных числу байт	Объем всей регистрируемой программы и состав поставки указываются заявителем. Размер выборки исходных файлов этого описания не является объемом программы.
7	Дата окончательного завершения программы (число, месяц, год)	Дата окончательного завершения программы указывается заявителем. 29.10.2026 – дата подготовки описания, не подтверждение завершения разработки.
8	ФИО каждого автора; дата рождения; адрес местожительства (включая индекс)	Заполняется заявителем: сведения не предоставлены.
9	В каком порядке	Заполняется заявителем: сведения не

№	Сведения по шаблону	Содержание для модуля «Экология»
	указывать ФИО авторов?	предоставлены.
10	Краткое описание авторского вклада в данную программу (Написание исходного текста программного модуля …; или Написание исходного текста подпрограммы …; или Разработка программ …; или Написание исходного текста программы – 00%)	Авторский вклад и доли определяются заявителем. Документ описывает реализацию модуля, но не устанавливает авторство конкретных лиц.
11	Исходный текст программы (листинг) (можно не подряд, а фрагментами). Допускается включать в состав этих мат-лов подготовительные материалы, полученные в ходе ее разработки, а также порождаемые ею аудиовизуальные отображения в любой визуально воспринимаемой форме. Экранные формы включаются в состав документов заявки в том случае, если они охраноспособны по нормам авторского права, т.е. являются результатом творческой деятельности автора.	Фрагменты исходного текста приведены в разделах 2–12, с указанием файлов и строк; экранные формы включены по месту описания. Представлены фрагменты, не полный текст.
12	Контактное лицо из числа авторов, тел.	Заполняется заявителем: сведения не предоставлены.

Сведения об авторах, их персональных данных, контактах, окончательном названии, объеме и дате завершения не определяются по имени пользователя в интерфейсе или по дате изменения файлов. Вопрос о включении экранных форм в состав охраняемых материалов решается заявителем с учетом прав на используемые элементы интерфейса.

2. Модуль «Экология»: назначение и архитектура

Модуль предназначен для хранения и просмотра отчетных данных о парниковых газах и загрязняющих веществах, выверки их относительно топливно-энергетических показателей, расчета выбросов действующих и новых ТЭС и подготовки сводных таблиц. Основной адрес: http://msk-arm-gs01.ntcees.ru/ecology/. В документе описывается текущая рабочая копия исходников; совпадение ее сборки с действующим сервером не установлено.

Различаются два контура. Отчетно-аналитический контур использует сведения ГИС ТЭК, выверенные параметры анализа и агрегационные суммы. Расчетный контур использует принятые топливные снимки, характеристики, коэффициенты и нормативы для действующих и новых ТЭС. Итоговые суммы анализа и сводные расчетные выбросы имеют разное происхождение и не заменяют друг друга.

2.1. Страницы, вкладки и границы описания

Описаны 30 функциональных экранных представлений: главная страница, 2 отчета, 4 листа анализа, 10 вкладок действующих ТЭС, 3 вкладки новых ТЭС, 4 сводные вкладки, 4 агрегированных листа, редактор подсказок и рассмотрение топливного пакета. Дополнительно перечислены 4 скрытые совместимые вкладки сводного расчета. Это не 30 отдельных HTML-шаблонов: используются 7 полноценных шаблонов с общими включаемыми фрагментами. Территория, год, мощностная категория, фильтр и пагинация являются параметрами представления. Все 34 объявления маршрутов, включая служебные операции и варианты адресов со слешем, перечислены в разделе 12.

На каждой странице приведены назначение, доступные действия, источники данных, связи с другими модулями и фрагменты реализации с точными путями и строками. Для страниц без отдельной модели указано отсутствие собственного хранилища. Фрагменты являются выборочным листингом, а не полным исходным текстом модуля.

2.2. Программные слои и модели данных

Flask Blueprint ecology_bp связывает маршруты с сервисами. ORM-модели SQLAlchemy хранят первичные и выверенные данные, а шаблоны Jinja и JavaScript выводят таблицы, фильтры, подсказки и интерактивный ввод. Для исходных числовых полей используется Numeric(36, 16); расчетные сервисы применяют Decimal и отдельные правила отображения. Округление для вывода не следует трактовать как изменение сохраненной точности.

Модель	Таблица	Ключи и разрезы
OperatingTesFormulaText	gs_eco_operating_tes_formula_texts	formula_key
EcologyFuelSnapshot	gs_eco_fuel_snapshots	database_version_id, year_number, scope
EcologyFuelSubmission	gs_eco_fuel_submissions	database_version_id, year_number, scope, status
NewTesNorm	gs_eco_new_tes_norms	database_version_id, year_number, fuel_key, capacity_class
OperatingTesCoefficient	gs_eco_operating_tes_coefficients	database_version_id, year_number, territory_slug, affiliation, kind

Модель	Таблица	Ключи и разрезы
OperatingTesFuelCharacteristic	gs_eco_operating_tes_fuel_chars	database_version_id, year_number, territory_slug, fuel_key
OperatingTesInputValue	gs_eco_operating_tes_inputs	database_version_id, year_number, territory_slug, row_key
PGParamCalc	gs_eco_analysis_pg_params	database_version_id, sheet_slug, station_numb, year_number
PGReportGisTek	gs_eco_gis_tek_greenhouse_gas_params	database_version_id, station_numb, equipment_group_id, year_number
ZVParamCalc	gs_eco_analysis_zv_params	database_version_id, sheet_slug, station_numb, year_number
ZVReportGisTek	gs_eco_gis_tek_pollutant_params	database_version_id, station_numb, equipment_group_id, year_number

Отчетные PGReportGisTek/ZVReportGisTek привязаны к коду группы оборудования, году и версии БД; отдельный equipment_group_id обеспечивает связь с группой «Топлива». Код numb не уникален сам по себе, поэтому версия и разрешение связи существенны. PGParamCalc/ZVParamCalc дополнительно разделены по sheet_slug. Составной внешний ключ года учитывает Year.number и Year.database_version_id. EcologyFuelSnapshot уникален по версии, году и scope. OperatingTesFormulaText, в отличие от числовых моделей, не имеет поля database_version_id: это общее хранилище текстов подсказок.

Модель отчетных ПГ: уникальность и составной ключ года. Источник: app/ecology/models/pg_report_gis_tek_model.py, строки 19–37.

```
19      __tablename__ = "gs_eco_gis_tek_greenhouse_gas_params"
20      __table_args__ = (
21          UniqueConstraint(
22              "station_numb",
23              "year_number",
24              "database_version_id",
25              name="uq_eco_gis_tek_pg_numb_year_ver",
26          ),
27          ForeignKeyConstraint(
28              ["year_number", "database_version_id"],
29              [
30                  f"{SCHEMA_REFDATA}.gs_sys_years.number",
31                  f"{SCHEMA_REFDATA}.gs_sys_years.database_version_id",
32              ],
33              ondelete="RESTRICT",
34              name="fk_eco_gis_tek_pg_year_ver",
35          ),
36          {"schema": SCHEMA_ECOLOGY},
37      )
```

Модель отчетных ПГ: исходные числовые поля. Источник: app/ecology/models/pg_report_gis_tek_model.py, строки 90–106.

```
90      # Расход газа, т у.т.
91      gas_consumption_tce = db.Column(Numeric(36, 16), nullable=True)
92
93      # Выбросы CO2 от сжигания газа, т
94      co2_from_gas_t = db.Column(Numeric(36, 16), nullable=True)
95
96      # Расход мазута, т у.т.
97      oil_fuel_consumption_tce = db.Column(Numeric(36, 16), nullable=True)
98
99      # Выбросы CO2 от сжигания мазута, т
100     co2_from_oil_fuel_t = db.Column(Numeric(36, 16), nullable=True)
101
102     # Расход твердого топлива, т у.т.
103     solid_fuel_consumption_tce = db.Column(Numeric(36, 16), nullable=True)
104
105     # Выбросы CO2 от сжигания твердого топлива, т
106     co2_from_solid_fuel_t = db.Column(Numeric(36, 16), nullable=True)
```

Выверенные параметры анализа ЗВ: хранимые компоненты. Источник: app/ecology/models/zv_param_calc_model.py, строки 76–95.

```
76      # Твердые вещества, т (TVERD_V)
77      solid_substances = db.Column(Numeric(36, 16), nullable=True)
78
79      # Диоксид серы, т (SO2_gaz)
80      sulfur_dioxide = db.Column(Numeric(36, 16), nullable=True)
81
82      # Оксид углерода [CO], т (CO_gaz)
83      carbon_monoxide = db.Column(Numeric(36, 16), nullable=True)
84
85      # Оксиды азота [в пересчете на NO2], т (NOx_gaz)
86      nitrogen_oxides = db.Column(Numeric(36, 16), nullable=True)
87
88      # Углеводороды [без летучих органических соединений], т (UGLEVOD)
89      hydrocarbons = db.Column(Numeric(36, 16), nullable=True)
90
91      # Летучие органические соединения [ЛОС], т (LOS)
92      voc = db.Column(Numeric(36, 16), nullable=True)
93
94      # В т.ч. прочие газообразные и жидкие, т (PROCH_ekol)
95      other_gaseous_and_liquid = db.Column(Numeric(36, 16), nullable=True)
```

Модель принятого топливного снимка. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
9   class EcologyFuelSnapshot(db.Model, AuditMixin):
10      __tablename__ = "gs_eco_fuel_snapshots"
11      __table_args__ = (
12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
13          {"schema": SCHEMA_ECOLOGY},
14      )
15
16      id = db.Column(db.Integer, primary_key=True)
17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
18      year_number = db.Column(db.Integer, nullable=False)
19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
20      payload = db.Column(db.JSON, nullable=False)
21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

2.3. Права, версия БД и журнал

Все внешние маршруты защищены login_required. Для записи can_edit_ecology требует аутентификацию и признак has_admin либо роль, имя которой содержит ecology. Отдельная проверка административной роли регулирует видимость служебных идентификационных столбцов. Недостаток прав блокирует импорт, PATCH, принятие пакета и сохранение текстов на сервере, независимо от того, показана ли кнопка в HTML.

Выбор версии БД используется в запросах отчетов, параметров и снимков. Передача топлива без выбранной версии завершается ошибкой. Журнал «Экология» регистрирует посещения, импорт и выгрузки, изменение значений и решения по пакетам. Дата, пользователь и контекст действия фиксируются существующими сервисами приложения; настоящее описание не подменяет эти записи.

Проверка права изменения экологических данных. Источник: app/ecology/services/gis_tek_page_services.py, строки 115–121.

```
115  def can_edit_ecology(user) -> bool:
116      if not user or not getattr(user, "is_authenticated", False):
117          return False
118      if getattr(user, "has_admin", False):
119          return True
120      names = [str(n or "") for n in (getattr(user, "role_names", None) or [])]
121      return any("ecology" in n.lower() for n in names)
```

Регистрация просмотра страницы. Источник: app/ecology/services/ecology_logging_services.py, строки 184–201.

```
184  def log_page_visit(
185      page_title: str,
186      *,
187      tab: Optional[str] = None,
188      details: Optional[Iterable[str]] = None,
189  ) -> None:
190      """Открытие страницы или вкладки модуля (дубль middleware подавляется)."""
191      action = f"{PAGE_VIEW_ACTION_PREFIX}: {MODULE_TITLE} – {page_title}"
192      if tab:
193          action += f", вкладка «{tab}»"
194      parts: list[str] = []
195      if has_request_context():
196          parts.append(f"url={request.full_path}")
197      parts.append(f"раздел={MODULE_TITLE}")
198      if tab:
199          parts.append(f"вкладка={tab}")
200      parts.extend(item for item in (details or ()) if item)
201      log_to_db(_actor(), action, "; ".join(parts), entity_type=ENTITY_TYPE)
```

2.4. Условия получения экранных форм

Снимки получены 29 октября 2026 года с действующего сайта. Для четырех аналитических листов выбран 2024 год. Общая панель с именем пользователя исключена кадрированием; широкие таблицы представлены видимым фрагментом. По согласованию с пользователем страницы анализа были открыты, хотя GET может создать недостающие строки PGParamCalc/ZVParamCalc из отчетов ГИС ТЭК. В рабочей БД могли появиться такие строки; их количество отдельно не проверялось. Значения существующих показателей не редактировались, формы импорта, PATCH, сохранение подсказок и решения по пакетам не отправлялись. Посещения могут регистрироваться штатным журналом приложения.

Просмотрены также главная страница, два отчета, редактор текстов, принятые топливные показатели действующих и новых ТЭС и коэффициенты. Другие страницы, GET которых вызывает ensure_default_persisted_rows или ensure_default_norm_rows, для иллюстраций не открывались. Их поведение описано по исходникам, без утверждения о проверке изменяющих операций на рабочей БД.

Расчетный GET может сохранять недостающие значения по умолчанию. Источник: app/ecology/services/operating_tes_page_services.py, строки 812–818.

```
812      elif layout == "chars":
813          ensure_default_persisted_rows(years)
814          char_overrides = load_characteristic_overrides(years)
815      else:
816          inputs = load_accepted_fuel_inputs(years)
817          ensure_default_persisted_rows(years)
818          input_overrides = {} if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"} else
load_input_overrides(years)
```

Разделение безопасного просмотра топлива и инициализации нормативов. Источник: app/ecology/services/new_tes_page_services.py, строки 275–291.

```
    char_overrides = load_characteristic_overrides(years) if layout != "fuel" else {}
    if layout == "fuel":
        raw = load_accepted_new_tes_fuel_inputs(years)
        inputs = {
            year: {
                slug: by_class.get(capacity_class) or empty_fuel_bucket()
                for slug, by_class in (by_terr or {}).items()
            }
            for year, by_terr in raw.items()
        }
    elif layout == "chars":
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
    else:
        raw = load_accepted_new_tes_fuel_inputs(years)
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
```

Инициализация параметров анализа при формировании годовых строк. Источник: app/ecology/services/gis_tes_pg_from_fuel_services.py, строки 790–810.

```
    slug = str(sheet_slug or "").strip()
    if slug not in ANALYSIS_SHEET_SLUGS or not groups:
        return
    years: list[int] = list(range(int(start_year), int(end_year) + 1))
    numbs: set[int] = set()
    for group in groups:
        for numb in group.get("member_numbs") or [group.get("numb")]:
            if isinstance(numb, int):
                numbs.add(numb)
    if slug in PG_ANALYSIS_SHEET_SLUGS:
        seed_missing_analysis_pg_params_from_reports(
            slug,
            station_numbs=numbs or None,
            years=years,
        )
    elif slug in ZV_ANALYSIS_SHEET_SLUGS:
        seed_missing_analysis_zv_params_from_reports(
            slug,
            station_numbs=numbs or None,
            years=years,
        )
```

3. Главная страница и отчетные данные ГИС ТЭК

3.1. Главная страница модуля

Адрес: /ecology/

Функциональные возможности и действия пользователя

Страница объединяет четыре группы переходов: отчетные данные, анализ данных, расчетные значения и агрегированные значения. Отдельные кнопки открывают ПГ и ЗВ, расчет действующих и новых ТЭС, сводные расчеты и итоговые суммы. Для пользователя с правом редактирования доступен редактор текстов подсказок формул. Главная страница не содержит собственной формы ввода экологических показателей.

Взаимодействие и источники данных

Навигация ведет к отчетным и расчетным подсистемам «Экологии». Общая панель связывает модуль с главной страницей АРМ, справочниками, версиями БД и журналом. Состав кнопок анализа формируется по COMPARISON_GROUPS, а доступ к редактору определяется can_edit_ecology.



Рисунок 1. Главная страница модуля «Экология». Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Собственной модели этой страницы нет: маршрут собирает навигацию и права, не создавая отдельную таблицу показателей.

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 141–149.

```
@ecology_bp.route("/")
@login_required
def hub():
    log_page_visit("Главная страница модуля")
    return render_template(
        "ecology/ecology_start.html",
        can_edit_ecology=can_edit_ecology(current_user),
        comparison_groups=COMPARISON_GROUPS,
    )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tek_page_services.py, строки 115–121.

```
def can_edit_ecology(user) -> bool:
    if not user or not getattr(user, "is_authenticated", False):
        return False
    if getattr(user, "has_admin", False):
        return True
    names = [str(n or "") for n in (getattr(user, "role_names", None) or [])]
    return any("ecology" in n.lower() for n in names)
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/ecology_start.html, строки 9–42.

```
  <section class="home-outline__section">
    <h2 class="home-outline__section-title">Отчетные данные</h2>
    <div class="home-outline__tiles">
      {{ hub_tile(url_for('ecology_bp.gis_tek_pg'), 'Парниковые
      газы (ПГ)') }} {{ hub_tile(url_for('ecology_bp.gis_tek_pollutants'),
      'Загрязняющие вещества (ЗВ)') }}
    </div>
  </section>

  <section class="home-outline__section">
    <h2 class="home-outline__section-title">Анализ данных</h2>
    <div class="home-outline__tiles">
      {% for group in comparison_groups %} {% call
      hub_tile(url_for('ecology_bp.gis_tes_fuel_comparison',
      sheet_slug=group.default_sheet_slug)) %} {% if group.slug == 'pg' %}
      Парниковые газы (ПГ)<br />анализ отчетных данных {% elif group.slug ==
      'zv' %} Загрязняющие вещества (ЗВ)<br />анализ отчетных данных {% else %}
      {{ group.title }} {% endif %} {% endcall %} {% endfor %}
    </div>
  </section>

  <section class="home-outline__section">
    <h2 class="home-outline__section-title">Расчетные значения</h2>
    <div class="home-outline__tiles">
      {{ hub_tile(url_for('ecology_bp.operating_tes_table',
      territory_slug='vsego', sheet_slug='obemy'), 'Расчет по действующим ТЭС',
      'is-accent') }} {{ hub_tile(url_for('ecology_bp.new_tes_table',
      territory_slug='vsego', sheet_slug='obemy'), 'Расчет по новым ТЭС',
      'is-accent') }}
    </div>
    <div class="home-outline__tiles mt-2">
      {{ hub_tile(url_for('ecology_bp.calculation_summary_index'),
      'Сводные данные расчетов', 'is-accent') }}
    </div>
```

3.2. Парниковые газы: отчетные данные ГИС ТЭС

Адрес: /ecology/gis_tek/pg/

Функциональные возможности и действия пользователя

Таблица показывает отчетные показатели по группам оборудования за выбранный год. Строки организованы по ОЭС и региональным энергосистемам. Доступны поиск по станции, UUID и компании, территориальные фильтры, отбор по принадлежности и коду numb1120, фильтры в шапке, пагинация, показ пустых строк, округление и суммы по регионам. Для газа, мазута и твердого топлива показаны расход в т у.т. и масса CO2 в тоннах. Два итоговых столбца суммируют

расход топлива и выбросы по трем группам; они вычисляются программой, а не загружаются как независимые исходные показатели. Панель импорта и экспорта содержит загрузку Excel для уполномоченного пользователя и выгрузку текущей выборки. Самостоятельной страницы добавления или карточки редактирования отчета нет.

Взаимодействие и источники данных

Группа оборудования и ее код поступают из «Топлива», наименование станции и ссылка на карточку берутся из «Генерации». Год и территория согласуются со справочниками и выбранной версией БД. Отчетные строки используются для первоначального заполнения отдельных параметров анализа; уже исправленные параметры анализа не должны автоматически заменяться повторным просмотром отчета.

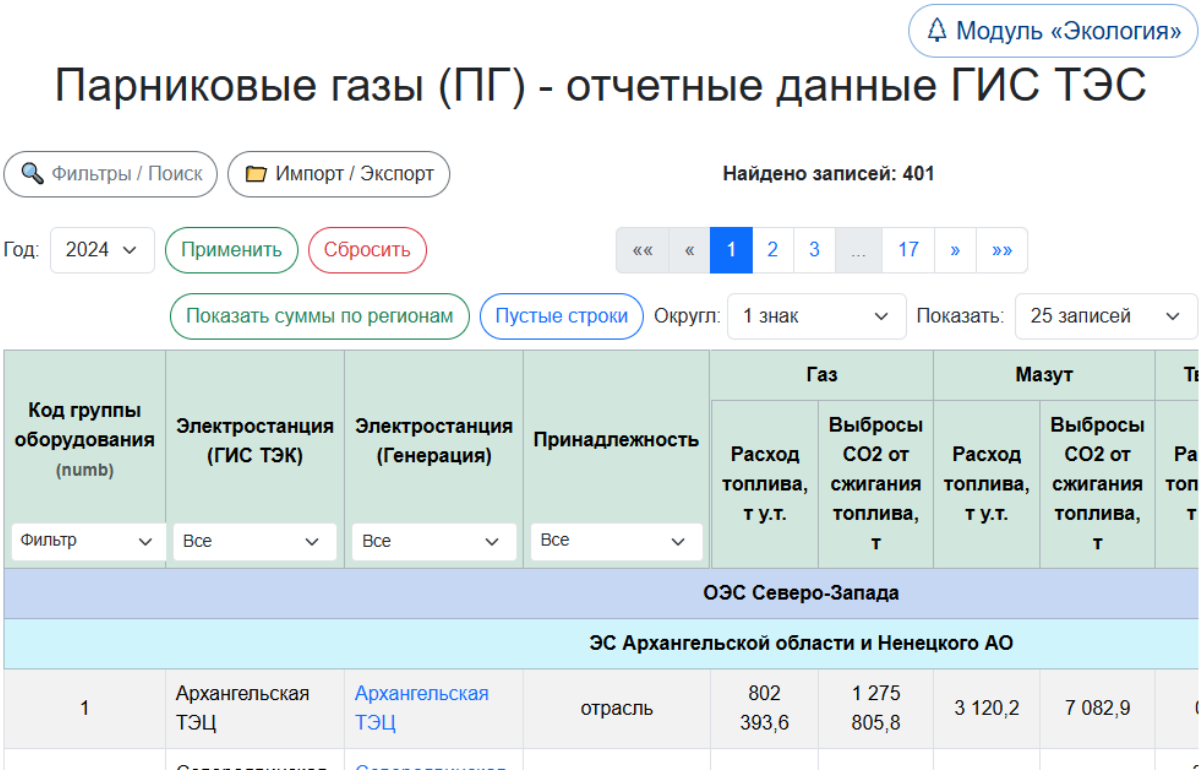


Рисунок 2. Парниковые газы: отчетные данные ГИС ТЭС. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

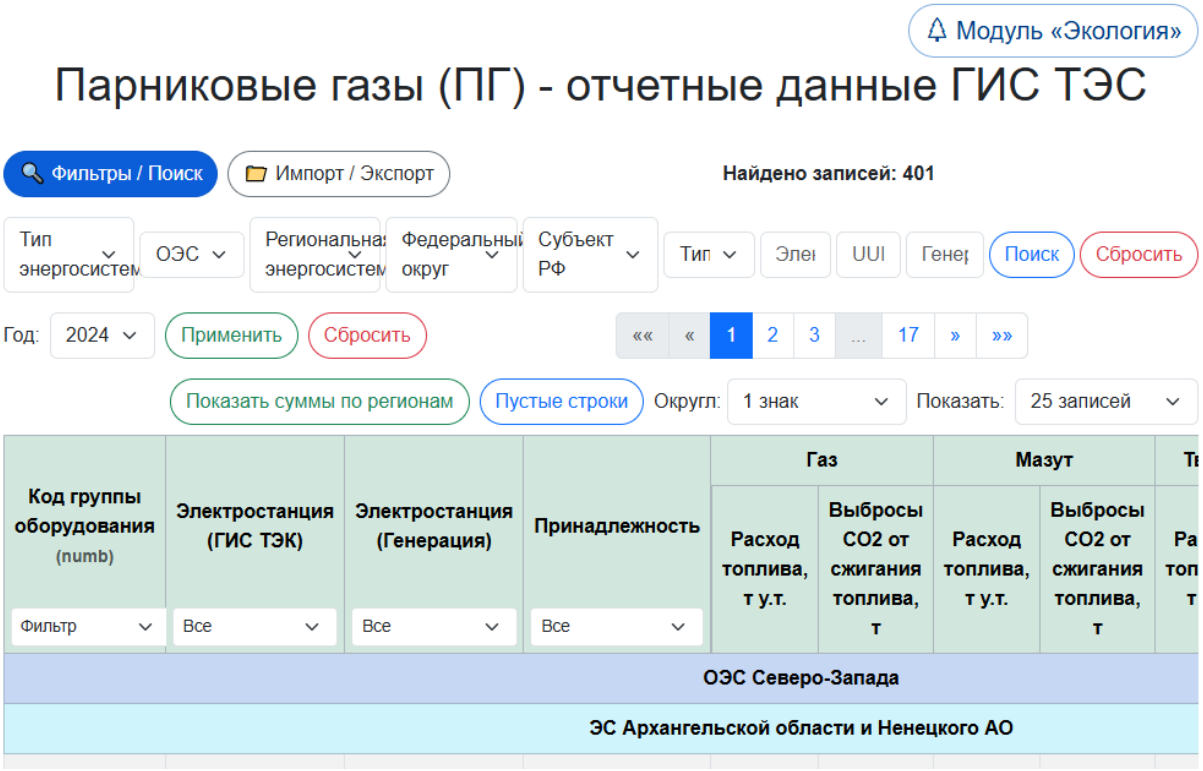


Рисунок 3. Раскрытая панель фильтров отчетных данных. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель PGReportGisTek. Источник: app/ecology/models/pg_report_gis_tek_model.py, строки 11–19.

```
11  class PGReportGisTek(db.Model, AuditMixin):
12      """Строка листа «СБОРКА ПГ»: выбросы CO2 по видам топлива и году.
13
14      Связь с EquipmentGroup через equipment_group_id и station_numb = EquipmentGroup.numb
15      (без FK на numb – поле не уникально).
16      year_number + database_version_id → Year.number + Year.database_version_id.
17      """
18
19      __tablename__ = "gs_eco_gis_tek_greenhouse_gas_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 219–222.

```
219  @ecology_bp.route("/gis_tek/pg/", methods=["GET"])
220  @login_required
221  def gis_tek_pg():
222      return _render_report(REPORT_PG)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tek_page_services.py, строки 621–626.

```
621  def apply_pg_derived_metrics(values: dict) -> dict:
622      """Суммы расхода топлива и выбросов CO2 по всем видам топлива."""
623      values = dict(values)
624      for attr, parts in PG_TOTAL_PARTS.items():
625          values[attr] = _sum_parts(values, parts)
626      return values
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/gis_tek_report.html, строки 228–257.

```
            </thead>
            <tbody>
                {% if hierarchy and hierarchy|length > 0 %}
                    {% for ues_block in hierarchy %}
                        <tr class="table-primary text-center" style="vertical-align: middle; font-
weight: bold;">
                            <td colspan="{{ column_count }}">{{ ues_block.ues_name }}</td>
                        </tr>
                        {% for res_block in ues_block.res_list %}
                            <tr class="table-info text-center" style="vertical-align: middle; font-
weight: bold;">
                                <td colspan="{{ column_count }}">{{ res_block.res_name }}</td>
                            </tr>
                            {% for row in res_block.rows %}
                            <tr class="text-center align-middle{% if row.is_empty %} ot-row-empty-
hidden{% endif %}">
                                {% if show_numb_column %}
                                <td>{{ row.station_numb if row.station_numb is not none else '–'
}}</td>
                                {% endif %}
                                <td class="text-start">{{ row.gis_tek_station_name }}</td>
                                <td class="text-start">
                                    {% if row.generation_stations %}
                                        {% for st in row.generation_stations %}
                                            {% if not loop.first %}<br>{% endif %}
                                            <a href="{{ url_for('station_bp.station_details',
station_id=st.id) }}"
                                               class="text-decoration-none"
                                               target="_blank"
                                               rel="noopener noreferrer">{{ st.name }}</a>
                                        {% endfor %}
                                    {% else %}
                                        –
                                    {% endif %}
                                </td>
```

Сервис формирования страницы. Источник: app/ecology/services/gis_tek_page_services.py, строки 942–970.

```
def build_gis_tek_page_context(kind: str, args, user=None) -> dict:
    model = REPORT_MODELS[kind]
    version_id = get_current_db_version_id()
    years = _filter_years()
    data_years = _data_years(model, version_id)
    if data_years:
        default_year = data_years[-1]
    else:
        from config import START_YEAR
        default_year = START_YEAR if START_YEAR in years else (years[-1] if years else None)
    selected_year = _resolve_year(args, years, default_year)

    per_page = _parse_per_page(args.get("per_page"))
    try:
        page = int(args.get("page") or 1)
    except (TypeError, ValueError):
        page = 1
    if page < 1:
        page = 1

    q = (args.get("q") or "").strip()
    station_type_filter = (args.get("station_type") or "").strip()
    numb1120_filter = normalize_numb_filter(_args_getlist(args, "numb1120_filter"))
    territory_filters = _parse_gis_tek_territory_filters(args)
    q = str(territory_filters.get("station_name_filter") or q or "").strip()
    show_totals = parse_show_totals_arg(args)
    show_empty_rows = parse_show_empty_rows_arg(args)
    rounding_digits = resolve_rounding_digits(args.get("rounding_digits"))
    norm_zv_selected = parse_norm_zv_parts(args) if kind == REPORT_ZV else []
```

Общий маршрут вывода отчета. Источник: app/ecology/routes/ecology_routes.py, строки 80–91.

```
80      context["import_endpoint"] = (
81          "ecology_bp.gis_tek_pg_import"
82          if kind == REPORT_PG
83          else "ecology_bp.gis_tek_pollutants_import"
84      )
85      context["export_endpoint"] = (
86          "ecology_bp.gis_tek_pg_export"
87          if kind == REPORT_PG
88          else "ecology_bp.gis_tek_pollutants_export"
89      )
90      log_page_visit(context["page_title"])
91      return render_template("ecology/gis_tek_report.html", **context)
```

3.3. Загрязняющие вещества: отчетные данные ГИС ТЭС

Адрес: /ecology/gis_tek/pollutants/

Функциональные возможности и действия пользователя

Таблица показывает отчетные показатели по группам оборудования за выбранный год. Строки организованы по ОЭС и региональным энергосистемам. Доступны поиск по станции, UUID и компании, территориальные фильтры, отбор по принадлежности и коду numb1120, фильтры в шапке, пагинация, показ пустых строк, округление и суммы по регионам. Показаны твердые вещества, диоксид серы, CO, оксиды азота, углеводороды, ЛОС и прочие газообразные и жидкие вещества в тоннах за год. Газообразные и жидкие вещества суммируются из компонентов, общий итог складывается из них и твердых веществ. Состав столбца «Нормируемые ЗВ» выбирается в шапке; по умолчанию включены твердые вещества, SO2, CO и NOx. Панель импорта и экспорта содержит загрузку Excel для уполномоченного пользователя и выгрузку текущей выборки. Самостоятельной страницы добавления или карточки редактирования отчета нет.

Взаимодействие и источники данных

Группа оборудования и ее код поступают из «Топлива», наименование станции и ссылка на карточку берутся из «Генерации». Год и территория согласуются со справочниками и выбранной версией БД. Отчетные строки используются для первоначального заполнения отдельных параметров анализа; уже исправленные параметры анализа не должны автоматически заменяться повторным просмотром отчета.

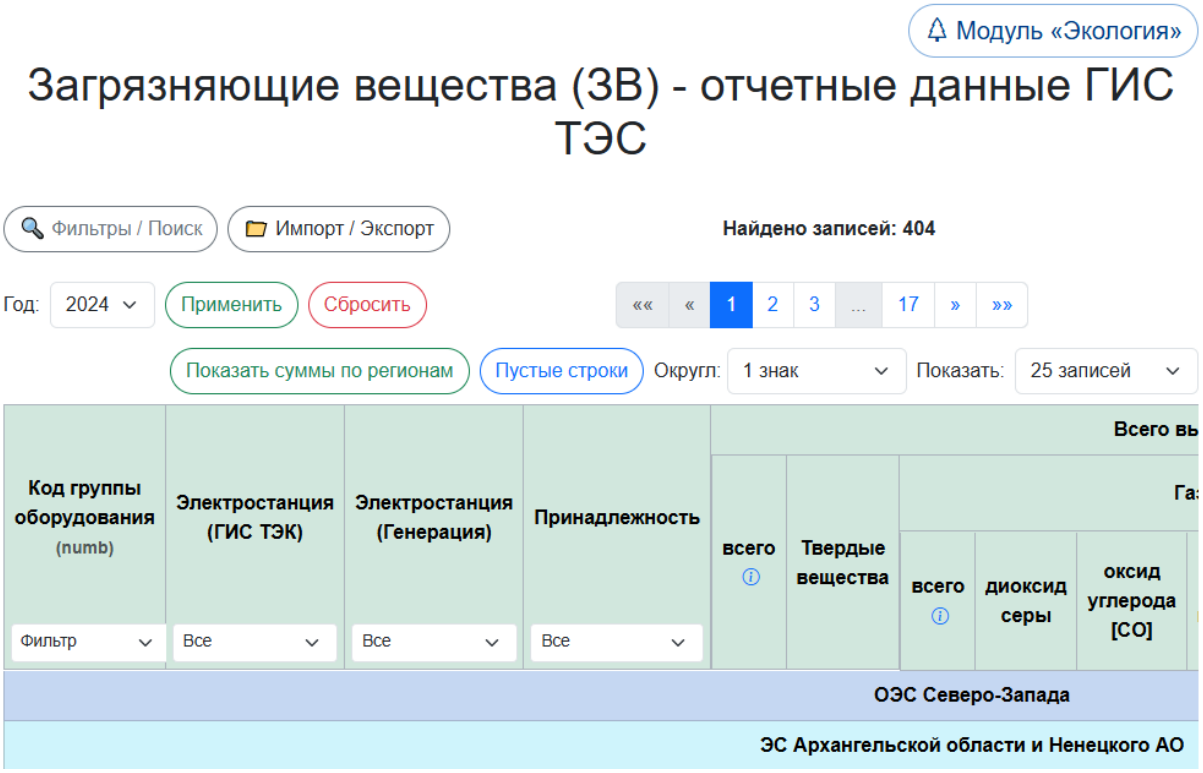


Рисунок 4. Загрязняющие вещества: отчетные данные ГИС ТЭС. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель ZVReportGisTek. Источник: app/ecology/models/zv_report_gis_tek_model.py, строки 11–19.

```
11  class ZVReportGisTek(db.Model, AuditMixin):
12      """Строка листа «СБОРКА ЗВ»: выбросы загрязняющих веществ по году.
13
14      Связь с EquipmentGroup через equipment_group_id и station_numb = EquipmentGroup.numb
15      (без FK на numb – поле не уникально).
16      year_number + database_version_id → Year.number + Year.database_version_id.
17      """
18
19      __tablename__ = "gs_eco_gis_tek_pollutant_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 237–240.

```
237  @ecology_bp.route("/gis_tek/pollutants/", methods=["GET"])
238  @login_required
239  def gis_tek_pollutants():
240      return _render_report(REPORT_ZV)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tek_page_services.py, строки 613–618.

```
613  def apply_zv_derived_metrics(values: dict) -> dict:
614      """Подставляет расчетные столбцы ЗВ (газообразные и жидкие, всего)."""
615      values = dict(values or {})
616      values["gaseous_and_liquid"] = sum_gaseous_and_liquid(values)
617      values["total"] = sum_zv_total(values)
618      return values
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/gis_tek_report.html, строки 228–257.

```
            </thead>
            <tbody>
                {% if hierarchy and hierarchy|length > 0 %}
                    {% for ues_block in hierarchy %}
                        <tr class="table-primary text-center" style="vertical-align: middle; font-
weight: bold;">
                            <td colspan="{{ column_count }}">{{ ues_block.ues_name }}</td>
                        </tr>
                        {% for res_block in ues_block.res_list %}
                            <tr class="table-info text-center" style="vertical-align: middle; font-
weight: bold;">
                                <td colspan="{{ column_count }}">{{ res_block.res_name }}</td>
                            </tr>
                            {% for row in res_block.rows %}
                            <tr class="text-center align-middle{% if row.is_empty %} ot-row-empty-
hidden{% endif %}">
                                {% if show_numb_column %}
                                <td>{{ row.station_numb if row.station_numb is not none else '–'
}}</td>
                                {% endif %}
                                <td class="text-start">{{ row.gis_tek_station_name }}</td>
                                <td class="text-start">
                                    {% if row.generation_stations %}
                                        {% for st in row.generation_stations %}
                                            {% if not loop.first %}<br>{% endif %}
                                            <a href="{{ url_for('station_bp.station_details',
station_id=st.id) }}"
                                               class="text-decoration-none"
                                               target="_blank"
                                               rel="noopener noreferrer">{{ st.name }}</a>
                                        {% endfor %}
                                    {% else %}
                                        –
                                    {% endif %}
                                </td>
```

Сервис формирования страницы. Источник: app/ecology/services/gis_tek_page_services.py, строки 942–970.

```
def build_gis_tek_page_context(kind: str, args, user=None) -> dict:
    model = REPORT_MODELS[kind]
    version_id = get_current_db_version_id()
    years = _filter_years()
    data_years = _data_years(model, version_id)
    if data_years:
        default_year = data_years[-1]
    else:
        from config import START_YEAR
        default_year = START_YEAR if START_YEAR in years else (years[-1] if years else None)
    selected_year = _resolve_year(args, years, default_year)

    per_page = _parse_per_page(args.get("per_page"))
    try:
        page = int(args.get("page") or 1)
    except (TypeError, ValueError):
        page = 1
    if page < 1:
        page = 1

    q = (args.get("q") or "").strip()
    station_type_filter = (args.get("station_type") or "").strip()
    numb1120_filter = normalize_numb_filter(_args_getlist(args, "numb1120_filter"))
    territory_filters = _parse_gis_tek_territory_filters(args)
    q = str(territory_filters.get("station_name_filter") or q or "").strip()
    show_totals = parse_show_totals_arg(args)
    show_empty_rows = parse_show_empty_rows_arg(args)
    rounding_digits = resolve_rounding_digits(args.get("rounding_digits"))
    norm_zv_selected = parse_norm_zv_parts(args) if kind == REPORT_ZV else []
```

Общий маршрут вывода отчета. Источник: app/ecology/routes/ecology_routes.py, строки 80–91.

```
80      context["import_endpoint"] = (
81          "ecology_bp.gis_tek_pg_import"
82          if kind == REPORT_PG
83          else "ecology_bp.gis_tek_pollutants_import"
84      )
85      context["export_endpoint"] = (
86          "ecology_bp.gis_tek_pg_export"
87          if kind == REPORT_PG
88          else "ecology_bp.gis_tek_pollutants_export"
89      )
90      log_page_visit(context["page_title"])
91      return render_template("ecology/gis_tek_report.html", **context)
```

4. Анализ отчетных данных и сопоставление с модулем «Топливо»

4.1. Парниковые газы: анализ с котельными

Адрес: /ecology/analysis/pg-vse/

Функциональные возможности и действия пользователя

Лист «pg-vse» предназначен для анализа с котельными. Сопоставляются расход газа, мазута и твердого топлива по топливным данным и выверенным данным экологии. Программа вычисляет разности, коэффициенты эмиссии CO2, суммарную массу ПГ и удельные выбросы на топливо и приведенную энергию. Отсутствующий знаменатель или ноль дают пустой расчетный результат, а не деление на ноль. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Состав включает группы котельных, участвующие в исходной выборке.

Взаимодействие и источники данных

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

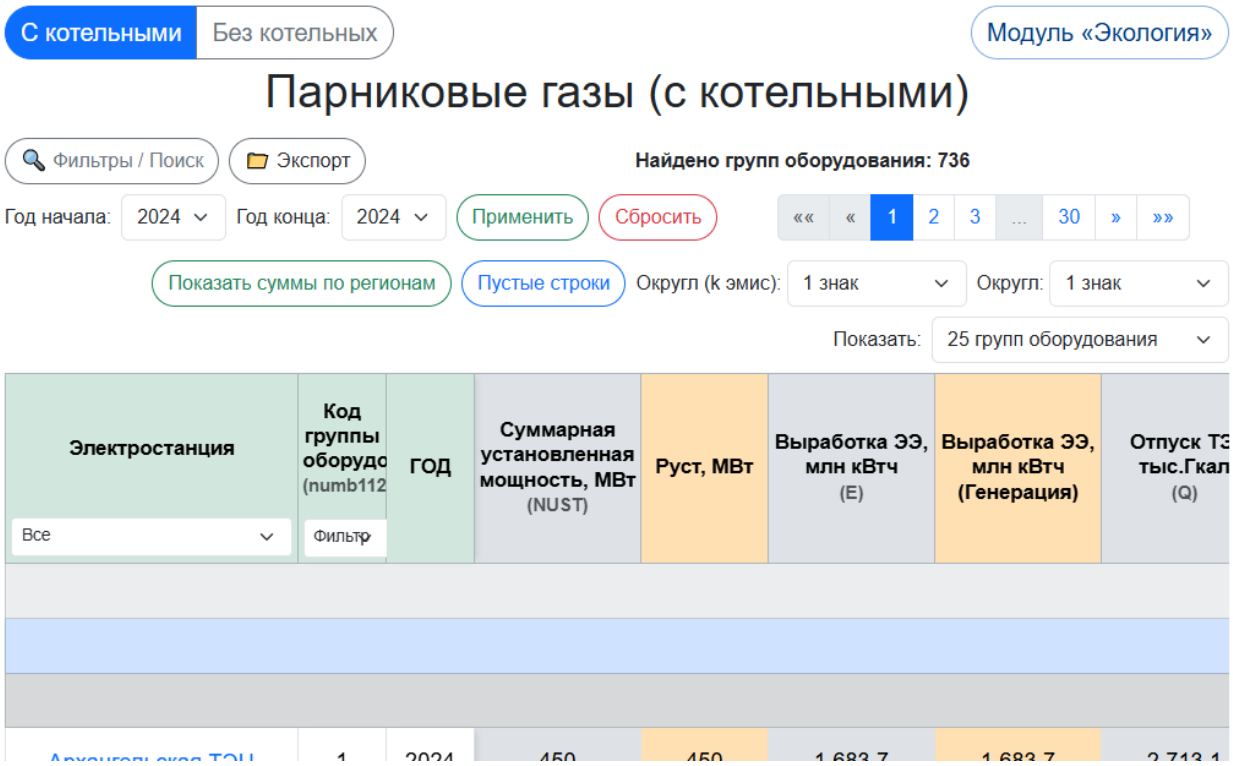


Рисунок 5. Парниковые газы – с котельными, 2024 год. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
class PGParamCalc(db.Model, AuditMixin):
    """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.

    Первоначально копируется из PGReportGisTek; далее может
    редактироваться отдельно от отчетных данных.
    Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
    (без FK на numb – поле не уникально).
    year_number + database_version_id → Year.number + Year.database_version_id.
    """

    __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
@ecology_bp.route("/analysis/<sheet_slug>/")
@login_required
def gis_tes_fuel_comparison(sheet_slug: str):
    if request.args.get("reset_filters") == "1":
        keep = {
            key: value
            for key in (
                "start_year",
                "end_year",
                "rounding_digits",
                "emission_rounding_digits",
                "specific_rounding_digits",
                "per_page",
                "show_totals",
                "show_empty_rows",
                "show_reported",
            )
            if (value := request.args.get(key)) not in (None, "")
        }
        if "norm_zv" in request.args:
            keep["norm_zv"] = request.args.getlist("norm_zv")
        return redirect(
            url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
        )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2028–2050.

```
def _compute_emission_coefficients(
    filled: list[Any],
    source_columns: list[dict[str, Any]],
) -> list[Any]:
    """Коэффициенты эмиссии: выбросы CO2 / расход топлива."""
    out = list(filled)
    for col_key, co2_key, fuel_key in [
        ("K_gaz_ek", "CO2_gaz", "GAZ_ek"),
        ("K_mazut_ek", "CO2_mazut", "MAZUT_ek"),
        ("K_ugol_ek", "CO2_ugol", "TvTopl_ek"),
    ]:
        idx = _column_index(source_columns, {col_key})
        if idx is None:
            continue
        co2_idx = _column_index(source_columns, {co2_key})
        fuel_idx = _column_index(source_columns, {fuel_key})
        if co2_idx is None or fuel_idx is None:
            continue
        value = _safe_divide(filled[co2_idx], filled[fuel_idx])
        if len(out) <= idx:
            out.extend([None] * (idx - len(out) + 1))
        out[idx] = value
    return out
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
 588            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
 589            id="gisTesFuelTable"
 590            data-sheet-slug="{{ sheet.slug }}"
 591            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
 592            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
 593            data-rounding-digits="{{ rounding_digits }}"
 594            data-emission-rounding-digits="{{ emission_rounding_digits }}"
 595            data-specific-rounding-digits="{{ specific_rounding_digits }}"
 596          >
 597            <thead class="table-success">
 598              {% for header_row in header_rows %}
 599              {% set row_is_key = header_row and header_row[0].is_key %}
 600              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
 601              {% for cell in header_row %}
 602              {% if cell.column_key == 'numb1120' and not cell.is_key %}
 603              {{ fuel_param_numb1120_header_th(
 604                   cell.rowspan if cell.rowspan else 1,
 605                   'Код группы оборудования',
 606                   numb1120_filter|default([]),
 607                   numb1120_filter_choices|default([]),
 608                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
 609                   sticky_col=cell.col_index
 610                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
 153  @ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
 154  @login_required
 155  def gis_tes_fuel_comparison_update_value(sheet_slug: str):
 156      if not can_edit_ecology(current_user):
 157          return jsonify(ok=False, error="Недостаточно прав"), 403
 158      if sheet_slug not in ANALYSIS_SHEET_SLUGS:
 159          return jsonify(ok=False, error="Лист только для просмотра"), 400
 160      data = request.get_json(silent=True) or {}
 161      column_key = str(data.get("field") or "").strip()
 162      try:
 163          station_numb = int(data.get("numb"))
 164          year_number = int(data.get("year"))
 165      except (TypeError, ValueError):
 166          return jsonify(ok=False, error="Некорректные станция или год"), 400
 167      raw = data.get("value")
 168      try:
 169          amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
 170      except Exception:
 171          return jsonify(ok=False, error="Некорректное число"), 400
 172      try:
 173          values = save_analysis_ecology_cell(
 174              sheet_slug,
 175              station_numb,
```

4.2. Парниковые газы: анализ без котельных

Адрес: /ecology/analysis/pg-bez-kot/

Функциональные возможности и действия пользователя

Лист «pg-bez-kot» предназначен для анализа без котельных. Сопоставляются расход газа, мазута и твердого топлива по топливным данным и выверенным данным экологии. Программа вычисляет разности, коэффициенты эмиссии CO2, суммарную массу ПГ и удельные выбросы на топливо и приведенную энергию. Отсутствующий знаменатель или ноль дают пустой расчетный результат, а не деление на ноль. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Из состава исключаются группы котельных; остальные показатели рассчитываются по той же схеме, что и на листе с котельными.

Взаимодействие и источники данных

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

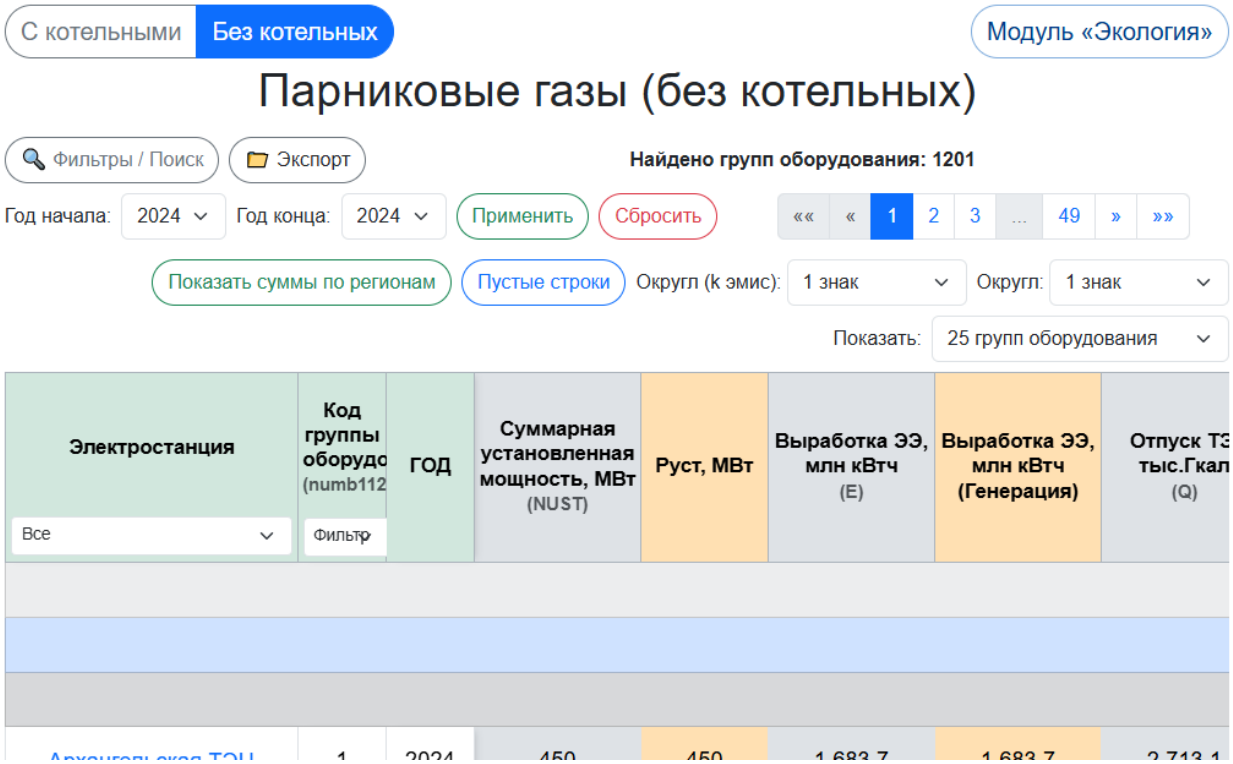


Рисунок 6. Парниковые газы – без котельных, 2024 год. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
11  class PGParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.
13
14      Первоначально копируется из PGReportGisTek; далее может
15      редактироваться отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
@ecology_bp.route("/analysis/<sheet_slug>/")
@login_required
def gis_tes_fuel_comparison(sheet_slug: str):
    if request.args.get("reset_filters") == "1":
        keep = {
            key: value
            for key in (
                "start_year",
                "end_year",
                "rounding_digits",
                "emission_rounding_digits",
                "specific_rounding_digits",
                "per_page",
                "show_totals",
                "show_empty_rows",
                "show_reported",
            )
            if (value := request.args.get(key)) not in (None, "")
        }
        if "norm_zv" in request.args:
            keep["norm_zv"] = request.args.getlist("norm_zv")
        return redirect(
            url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
        )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2028–2050.

```
def _compute_emission_coefficients(
    filled: list[Any],
    source_columns: list[dict[str, Any]],
) -> list[Any]:
    """Коэффициенты эмиссии: выбросы CO2 / расход топлива."""
    out = list(filled)
    for col_key, co2_key, fuel_key in [
        ("K_gaz_ek", "CO2_gaz", "GAZ_ek"),
        ("K_mazut_ek", "CO2_mazut", "MAZUT_ek"),
        ("K_ugol_ek", "CO2_ugol", "TvTopl_ek"),
    ]:
        idx = _column_index(source_columns, {col_key})
        if idx is None:
            continue
        co2_idx = _column_index(source_columns, {co2_key})
        fuel_idx = _column_index(source_columns, {fuel_key})
        if co2_idx is None or fuel_idx is None:
            continue
        value = _safe_divide(filled[co2_idx], filled[fuel_idx])
        if len(out) <= idx:
            out.extend([None] * (idx - len(out) + 1))
        out[idx] = value
    return out
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
 588            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
 589            id="gisTesFuelTable"
 590            data-sheet-slug="{{ sheet.slug }}"
 591            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
 592            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
 593            data-rounding-digits="{{ rounding_digits }}"
 594            data-emission-rounding-digits="{{ emission_rounding_digits }}"
 595            data-specific-rounding-digits="{{ specific_rounding_digits }}"
 596          >
 597            <thead class="table-success">
 598              {% for header_row in header_rows %}
 599              {% set row_is_key = header_row and header_row[0].is_key %}
 600              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
 601              {% for cell in header_row %}
 602              {% if cell.column_key == 'numb1120' and not cell.is_key %}
 603              {{ fuel_param_numb1120_header_th(
 604                   cell.rowspan if cell.rowspan else 1,
 605                   'Код группы оборудования',
 606                   numb1120_filter|default([]),
 607                   numb1120_filter_choices|default([]),
 608                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
 609                   sticky_col=cell.col_index
 610                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
 153  @ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
 154  @login_required
 155  def gis_tes_fuel_comparison_update_value(sheet_slug: str):
 156      if not can_edit_ecology(current_user):
 157          return jsonify(ok=False, error="Недостаточно прав"), 403
 158      if sheet_slug not in ANALYSIS_SHEET_SLUGS:
 159          return jsonify(ok=False, error="Лист только для просмотра"), 400
 160      data = request.get_json(silent=True) or {}
 161      column_key = str(data.get("field") or "").strip()
 162      try:
 163          station_numb = int(data.get("numb"))
 164          year_number = int(data.get("year"))
 165      except (TypeError, ValueError):
 166          return jsonify(ok=False, error="Некорректные станция или год"), 400
 167      raw = data.get("value")
 168      try:
 169          amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
 170      except Exception:
 171          return jsonify(ok=False, error="Некорректное число"), 400
 172      try:
 173          values = save_analysis_ecology_cell(
 174              sheet_slug,
 175              station_numb,
```

4.3. Загрязняющие вещества: анализ с котельными

Адрес: /ecology/analysis/zv-vse/

Функциональные возможности и действия пользователя

Лист «zv-vse» предназначен для анализа с котельными. Сопоставляются массы загрязняющих веществ, расход топлива, выработка электроэнергии и отпуск тепла. Вычисляются общий выброс, сумма нормируемых ингредиентов, приведенная энергия и удельные показатели на энергию и топливо. Можно менять состав нормируемых ЗВ и показывать дополнительные столбцы отчитавшихся станций. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Состав включает группы котельных, участвующие в исходной выборке.

Взаимодействие и источники данных

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

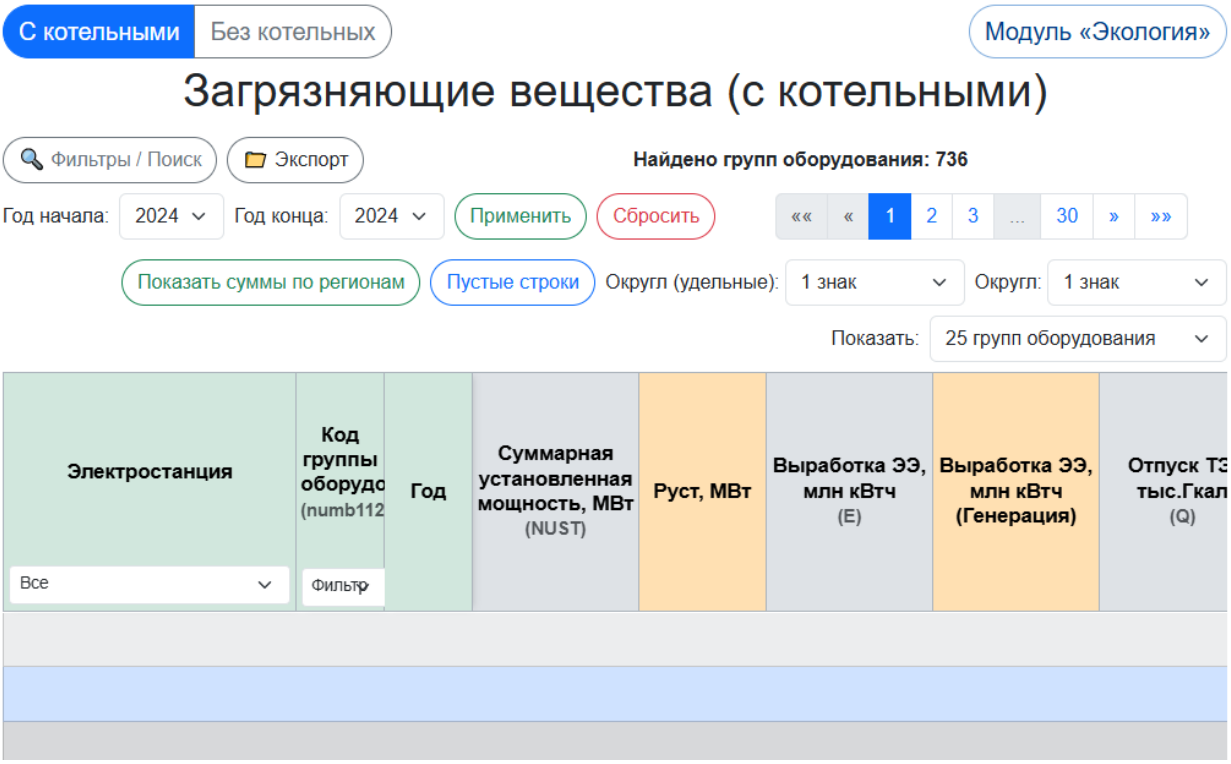


Рисунок 7. Загрязняющие вещества – с котельными, 2024 год. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
class ZVParamCalc(db.Model, AuditMixin):
    """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.

    Первоначально копируется из ZVReportGisTek; далее может редактироваться
    отдельно от отчетных данных.
    Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
    (без FK на numb – поле не уникально).
    year_number + database_version_id → Year.number + Year.database_version_id.
    """

    __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
@ecology_bp.route("/analysis/<sheet_slug>/")
@login_required
def gis_tes_fuel_comparison(sheet_slug: str):
    if request.args.get("reset_filters") == "1":
        keep = {
            key: value
            for key in (
                "start_year",
                "end_year",
                "rounding_digits",
                "emission_rounding_digits",
                "specific_rounding_digits",
                "per_page",
                "show_totals",
                "show_empty_rows",
                "show_reported",
            )
            if (value := request.args.get(key)) not in (None, "")
        }
        if "norm_zv" in request.args:
            keep["norm_zv"] = request.args.getlist("norm_zv")
        return redirect(
            url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
        )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2200–2222.

```
def _compute_zv_specific_metrics(
    filled: list[Any],
    source_columns: list[dict[str, Any]],
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> list[Any]:
    """Приведенная энергия и удельные ЗВ: на энергию, на выработку и на топливо."""
    out = list(filled)
    solid = _filled_at(out, source_columns, {ZV_SOLID_KEY})
    so2 = _filled_at(out, source_columns, {"SO2_gaz"})
    nox = _filled_at(out, source_columns, {"NOx_gaz"})
    zv_normir = _safe_sum(*(
        _filled_at(out, source_columns, {key})
        for key, attr in ZV_ECOLOGY_OVERLAY_BY_KEY.items()
        if attr in norm_zv_parts
    ))
    _assign_filled(out, source_columns, "ZV_normir", zv_normir)

    w_en = _filled_at(out, source_columns, {"W_en"})
    teplo = _filled_at(out, source_columns, {"Teplo"})
    energy = _brought_energy_mln_kwh(w_en, teplo)
    _assign_filled(out, source_columns, "energy", energy)

    total_zv = _filled_at(out, source_columns, {ZV_TOTAL_KEY})
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
 588            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
 589            id="gisTesFuelTable"
 590            data-sheet-slug="{{ sheet.slug }}"
 591            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
 592            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
 593            data-rounding-digits="{{ rounding_digits }}"
 594            data-emission-rounding-digits="{{ emission_rounding_digits }}"
 595            data-specific-rounding-digits="{{ specific_rounding_digits }}"
 596          >
 597            <thead class="table-success">
 598              {% for header_row in header_rows %}
 599              {% set row_is_key = header_row and header_row[0].is_key %}
 600              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
 601              {% for cell in header_row %}
 602              {% if cell.column_key == 'numb1120' and not cell.is_key %}
 603              {{ fuel_param_numb1120_header_th(
 604                   cell.rowspan if cell.rowspan else 1,
 605                   'Код группы оборудования',
 606                   numb1120_filter|default([]),
 607                   numb1120_filter_choices|default([]),
 608                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
 609                   sticky_col=cell.col_index
 610                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
 153  @ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
 154  @login_required
 155  def gis_tes_fuel_comparison_update_value(sheet_slug: str):
 156      if not can_edit_ecology(current_user):
 157          return jsonify(ok=False, error="Недостаточно прав"), 403
 158      if sheet_slug not in ANALYSIS_SHEET_SLUGS:
 159          return jsonify(ok=False, error="Лист только для просмотра"), 400
 160      data = request.get_json(silent=True) or {}
 161      column_key = str(data.get("field") or "").strip()
 162      try:
 163          station_numb = int(data.get("numb"))
 164          year_number = int(data.get("year"))
 165      except (TypeError, ValueError):
 166          return jsonify(ok=False, error="Некорректные станция или год"), 400
 167      raw = data.get("value")
 168      try:
 169          amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
 170      except Exception:
 171          return jsonify(ok=False, error="Некорректное число"), 400
 172      try:
 173          values = save_analysis_ecology_cell(
 174              sheet_slug,
 175              station_numb,
```

4.4. Загрязняющие вещества: анализ без котельных

Адрес: /ecology/analysis/zv-bez-kot/

Функциональные возможности и действия пользователя

Лист «zv-bez-kot» предназначен для анализа без котельных. Сопоставляются массы загрязняющих веществ, расход топлива, выработка электроэнергии и отпуск тепла. Вычисляются общий выброс, сумма нормируемых ингредиентов, приведенная энергия и удельные показатели на энергию и топливо. Можно менять состав нормируемых ЗВ и показывать дополнительные столбцы отчитавшихся станций. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Из состава исключаются группы котельных; остальные показатели рассчитываются по той же схеме, что и на листе с котельными.

Взаимодействие и источники данных

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

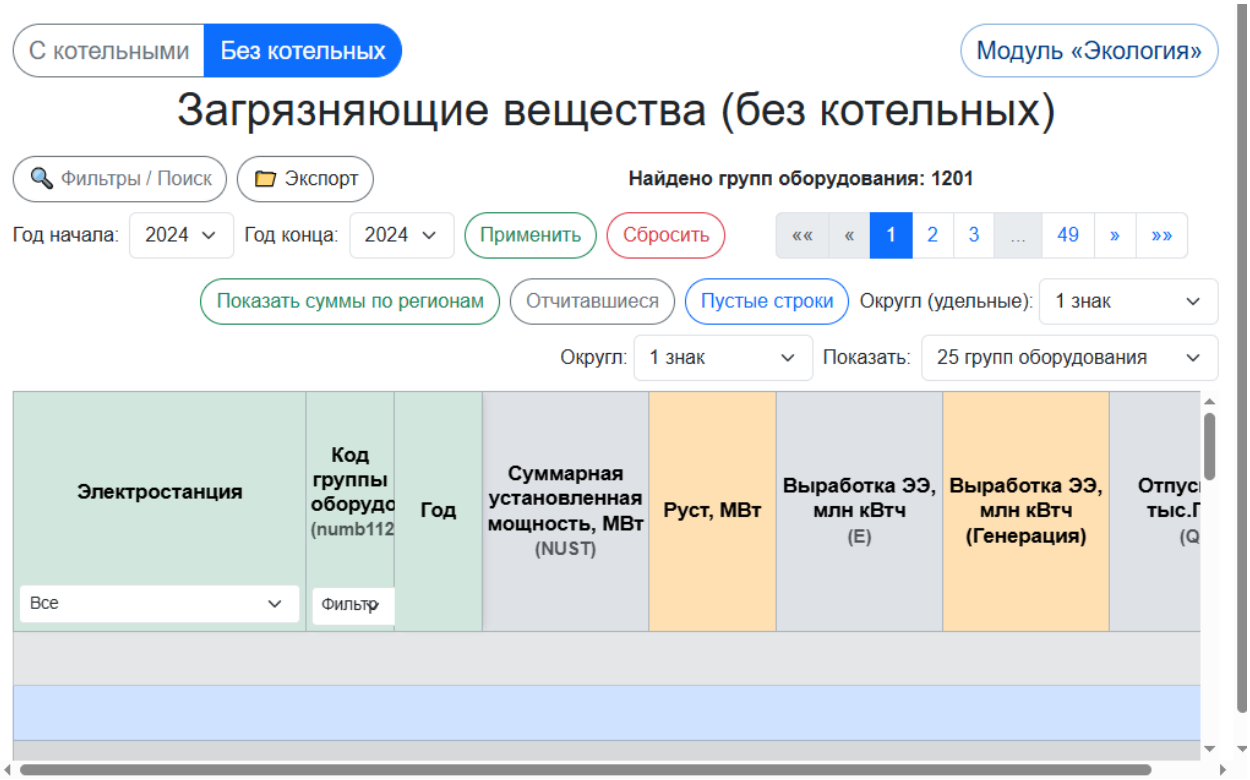


Рисунок 8. Загрязняющие вещества – без котельных, 2024 год. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
class ZVParamCalc(db.Model, AuditMixin):
    """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.

    Первоначально копируется из ZVReportGisTek; далее может редактироваться
    отдельно от отчетных данных.
    Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
    (без FK на numb – поле не уникально).
    year_number + database_version_id → Year.number + Year.database_version_id.
    """

    __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
@ecology_bp.route("/analysis/<sheet_slug>/")
@login_required
def gis_tes_fuel_comparison(sheet_slug: str):
    if request.args.get("reset_filters") == "1":
        keep = {
            key: value
            for key in (
                "start_year",
                "end_year",
                "rounding_digits",
                "emission_rounding_digits",
                "specific_rounding_digits",
                "per_page",
                "show_totals",
                "show_empty_rows",
                "show_reported",
            )
            if (value := request.args.get(key)) not in (None, "")
        }
        if "norm_zv" in request.args:
            keep["norm_zv"] = request.args.getlist("norm_zv")
        return redirect(
            url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
        )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2200–2222.

```
def _compute_zv_specific_metrics(
    filled: list[Any],
    source_columns: list[dict[str, Any]],
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> list[Any]:
    """Приведенная энергия и удельные ЗВ: на энергию, на выработку и на топливо."""
    out = list(filled)
    solid = _filled_at(out, source_columns, {ZV_SOLID_KEY})
    so2 = _filled_at(out, source_columns, {"SO2_gaz"})
    nox = _filled_at(out, source_columns, {"NOx_gaz"})
    zv_normir = _safe_sum(*(
        _filled_at(out, source_columns, {key})
        for key, attr in ZV_ECOLOGY_OVERLAY_BY_KEY.items()
        if attr in norm_zv_parts
    ))
    _assign_filled(out, source_columns, "ZV_normir", zv_normir)

    w_en = _filled_at(out, source_columns, {"W_en"})
    teplo = _filled_at(out, source_columns, {"Teplo"})
    energy = _brought_energy_mln_kwh(w_en, teplo)
    _assign_filled(out, source_columns, "energy", energy)

    total_zv = _filled_at(out, source_columns, {ZV_TOTAL_KEY})
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
 588            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
 589            id="gisTesFuelTable"
 590            data-sheet-slug="{{ sheet.slug }}"
 591            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
 592            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
 593            data-rounding-digits="{{ rounding_digits }}"
 594            data-emission-rounding-digits="{{ emission_rounding_digits }}"
 595            data-specific-rounding-digits="{{ specific_rounding_digits }}"
 596          >
 597            <thead class="table-success">
 598              {% for header_row in header_rows %}
 599              {% set row_is_key = header_row and header_row[0].is_key %}
 600              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
 601              {% for cell in header_row %}
 602              {% if cell.column_key == 'numb1120' and not cell.is_key %}
 603              {{ fuel_param_numb1120_header_th(
 604                   cell.rowspan if cell.rowspan else 1,
 605                   'Код группы оборудования',
 606                   numb1120_filter|default([]),
 607                   numb1120_filter_choices|default([]),
 608                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
 609                   sticky_col=cell.col_index
 610                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
 153  @ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
 154  @login_required
 155  def gis_tes_fuel_comparison_update_value(sheet_slug: str):
 156      if not can_edit_ecology(current_user):
 157          return jsonify(ok=False, error="Недостаточно прав"), 403
 158      if sheet_slug not in ANALYSIS_SHEET_SLUGS:
 159          return jsonify(ok=False, error="Лист только для просмотра"), 400
 160      data = request.get_json(silent=True) or {}
 161      column_key = str(data.get("field") or "").strip()
 162      try:
 163          station_numb = int(data.get("numb"))
 164          year_number = int(data.get("year"))
 165      except (TypeError, ValueError):
 166          return jsonify(ok=False, error="Некорректные станция или год"), 400
 167      raw = data.get("value")
 168      try:
 169          amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
 170      except Exception:
 171          return jsonify(ok=False, error="Некорректное число"), 400
 172      try:
 173          values = save_analysis_ecology_cell(
 174              sheet_slug,
 175              station_numb,
```

5. Расчет по действующим ТЭС

Территориальные разрезы обычных расчетных форм: Всего; станции отрасли; промпредприятия; ОЭС Северо-Запада, Центра, Средней Волги, Юга, Урала, Сибири и Востока; ТИТЭС Сибири и Востока; децентрализованная зона. На коэффициентах используются отдельные территориальные ключи tites-sibir, tites-vostok и decentralized-zone, а принадлежность выбирается параметром affiliation. Большинство годовых форм показывают территории столбцами, коэффициенты показывают выбранную территорию и годы столбцами.

Каталоги территорий и принадлежностей. Источник: app/ecology/services/operating_tes_constants.py, строки 20–47.

```
  20  TERRITORIES: tuple[dict[str, Any], ...] = (
  21      {'slug': 'vsego', 'sheet_name': 'Всего', 'group': 'territory'},
  22      {'slug': 'minen', 'sheet_name': 'Станции отрасли', 'group': 'special'},
  23      {'slug': 'prompred', 'sheet_name': 'Пром-предприятия', 'group': 'special'},
  24      {'slug': 'severo-zapad', 'sheet_name': 'ОЭС Северо-Запада', 'group': 'territory'},
  25      {'slug': 'centr', 'sheet_name': 'ОЭС Центра', 'group': 'territory'},
  26      {'slug': 'srednyaya-volga', 'sheet_name': 'ОЭС Средней Волги', 'group': 'territory'},
  27      {'slug': 'yug', 'sheet_name': 'ОЭС Юга', 'group': 'territory'},
  28      {'slug': 'ural', 'sheet_name': 'ОЭС Урала', 'group': 'territory'},
  29      {'slug': 'sibir', 'sheet_name': 'ОЭС Сибири', 'group': 'territory'},
  30      {'slug': 'vostok', 'sheet_name': 'ОЭС Востока', 'group': 'territory'},
  31      {'slug': 'norilsk', 'sheet_name': 'ТИТЭС Сибири', 'group': 'special'},
  32      {'slug': 'isolated-dv', 'sheet_name': 'ТИТЭС Востока', 'group': 'special'},
  33      {'slug': 'nikolaevsky', 'sheet_name': 'Децентрализованная зона (Николаевский энергорайон)',
'group': 'special'},
  34  )
  35
  36  COEFFICIENT_TERRITORIES: tuple[dict[str, Any], ...] = (
  37      *(item for item in TERRITORIES if item["slug"] not in {
  38          "minen", "prompred", "isolated-dv", "nikolaevsky", "norilsk"
  39      }),
  40      {"slug": "tites-sibir", "sheet_name": "ТИТЭС Сибири"},
  41      {"slug": "tites-vostok", "sheet_name": "ТИТЭС Востока"},
  42      {"slug": "decentralized-zone", "sheet_name": "Децентрализованная зона (Николаевский
энергорайон)"},
  43  )
  44  COEFFICIENT_AFFILIATIONS = (
  45      {"slug": "minen", "label": "Станции отрасли"},
  46      {"slug": "prompred", "label": "Промпредприятия"},
  47  )
```

5.1. Данные по модулю Топливо

Адрес: /ecology/operating_tes/vsego/toplivo/

Функциональные возможности и действия пользователя

Просмотр принятых топливных показателей действующих ТЭС: выработка электроэнергии, отпуск тепла, расход топлива и его видовой состав по территориям. Выбираются период и год; индикатор новых данных ведет к рассмотрению переданного пакета. Эта вкладка не разрешает прямую правку показателей. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

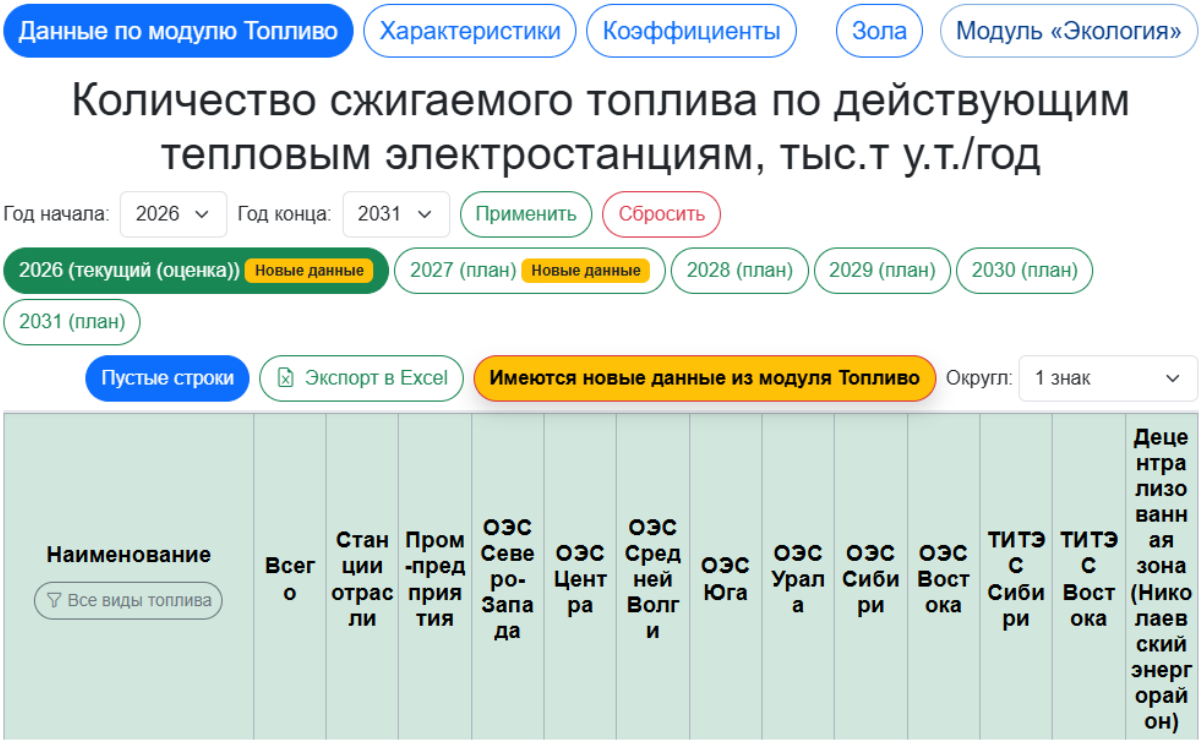


Рисунок 9. Принятые данные по модулю «Топливо» для действующих ТЭС. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
 61  @ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
 62  @login_required
 63  def operating_tes_table(territory_slug: str, sheet_slug: str):
 64      context = build_operating_tes_table_context(
 65          territory_slug,
 66          sheet_slug,
 67          start_year=request.args.get("start_year"),
 68          end_year=request.args.get("end_year"),
 69          rounding_digits=request.args.get("rounding_digits"),
 70          coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
 71          specific_rounding_digits=request.args.get("specific_rounding_digits"),
 72          view_year=request.args.get("view_year"),
 73          affiliation=request.args.get("affiliation", "all"),
 74      )
 75      if context is None:
 76          abort(404)
 77      sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/fuel_exchange_services.py, строки 34–41.

```
  34  def load_accepted_fuel_inputs(years, scope="operating"):
  35      rows = EcologyFuelSnapshot.query.filter_by(database_version_id=_version_id(),
scope=scope).filter(
  36          EcologyFuelSnapshot.year_number.in_(years)
  37      ).all()
  38      result = {year: {} for year in years}
  39      for row in rows:
  40          result[row.year_number] = decode_inputs(row.payload)
  41      return result
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 787–820.

```
 787            <th class="text-center align-middle ot-label-col">Наименование
 788              {% if sheet.layout in ('fuel', 'coeff') %}
 789              {% include 'ecology/operating_tes/_fuel_filter.html' %}
 790              {% endif %}
 791              {% if territory_filter_storage_key %}
 792              {% include 'ecology/operating_tes/_territory_filter.html' %}
 793              {% endif %}
 794            </th>
 795            {% for col in territory_columns %}
 796            <th class="text-center align-middle ot-terr-col">
 797              {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
 798              <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
 799              {% endif %}
 800            </th>
 801            {% endfor %}
 802          </tr>
 803        </thead>
 804        <tbody>
 805          {% for row in rows %}
 806          <tr
 807            class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
 808            data-row-key="{{ row.key }}"
 809            data-editable-field="{{ row.editable_field or '' }}"
 810          >
 811            <td
 812              class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
 813            >
 814              {{ row.label }}{% if row.key in ('ee_generation', 'heat_output') and row.unit %}, {{
row.unit }}{% endif %} {% if row.formula_tooltip %}
 815              <i
 816                class="bi bi-info-circle text-primary ms-1"
 817                data-bs-toggle="tooltip"
 818                data-bs-placement="top"
 819                {% if row.formula_tooltip_html %}data-bs-html="true" title="{{
row.formula_tooltip_html|forceescape }}"{% else %}title="{{ row.formula_tooltip|e }}"{% endif %}
 820                style="cursor: help; font-size: 0.9em"
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 789–798.

```
 789      if layout == "fuel":
 790          if fuel_source:
 791              from app.ecology.services.fuel_summary_services import build_fuel_summary
 792
 793              summary, errors = build_fuel_summary(years)
 794              inputs = summary[fuel_scope]
 795              fuel_summary_errors = errors[fuel_scope]
 796          else:
 797              inputs = load_accepted_fuel_inputs(years)
 798      elif layout == "coeff":
```

5.2. Характеристики топлив и продуктов сгорания

Адрес: /ecology/operating_tes/vsego/khar-topl/

Функциональные возможности и действия пользователя

Справочная таблица содержит коэффициент перевода условного топлива в натуральное, низшую теплоту сгорания, зольность, сернистость, доли уноса и связывания, потери от недожога и коэффициенты CO2. Уполномоченный

пользователь меняет значения по виду топлива и году. Характеристики общие для территорий и принадлежностей; локальный территориальный фильтр не создает отдельного набора физических свойств. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
61  @ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
62  @login_required
63  def operating_tes_table(territory_slug: str, sheet_slug: str):
64      context = build_operating_tes_table_context(
65          territory_slug,
66          sheet_slug,
67          start_year=request.args.get("start_year"),
68          end_year=request.args.get("end_year"),
69          rounding_digits=request.args.get("rounding_digits"),
70          coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
71          specific_rounding_digits=request.args.get("specific_rounding_digits"),
72          view_year=request.args.get("view_year"),
73          affiliation=request.args.get("affiliation", "all"),
74      )
75      if context is None:
76          abort(404)
77      sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_page_services.py, строки 383–399.

```
383  def resolve_chars(
384      fuel_key: str,
385      year: int,
386      overrides: dict[tuple[str, str, int], dict[str, Any]],
387      territory_slug: str = DEFAULT_TERRITORY_SLUG,
388  ) -> dict[str, Any]:
389      base = dict(DEFAULT_CHARACTERISTICS.get(fuel_key) or {})
390      if fuel_key in DEFAULT_CHAR_NOTES:
391          base["note"] = DEFAULT_CHAR_NOTES[fuel_key]
392      if fuel_key in DEFAULT_CHAR_SOURCES:
393          base["source_key"] = DEFAULT_CHAR_SOURCES[fuel_key]
394      # Характеристики общие для всех территорий и принадлежностей.
395      extra = overrides.get((DEFAULT_TERRITORY_SLUG, fuel_key, year)) or {}
396      for key, value in extra.items():
397          if value is not None:
398              base[key] = value
399      return base
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 548–584.

```
548        {% elif sheet.layout == 'chars' %}
549        <colgroup>
550          <col class="ot-col-label" />
551          {% for col in char_columns %}
552          <col />
553          {% endfor %}
554        </colgroup>
555        <thead class="table-success">
556          <tr class="text-center align-middle">
557            <th rowspan="2" scope="col" class="text-center align-middle ot-label-col">
558              Вид топлива
559              {% include 'ecology/operating_tes/_fuel_filter.html' %}
560            </th>
561            {% for group in char_header_groups %}
562            <th colspan="{{ group['keys']|length }}" scope="{{ 'colgroup' if group['keys']|length > 1
else 'col' }}"{% if group.rowspan %} rowspan="{{ group.rowspan }}"{% endif %} class="text-center align-
middle ot-char-title">
563              {{ group.title }}
564              {% if new_tes_capacity_tabs and 'k_co2' in group['keys'] %}
565              <button type="button" class="btn btn-link p-0 text-primary ms-1"
566                      data-bs-toggle="tooltip" data-bs-placement="top"
567                      title="Коэффициент ПГ общий с КСО2 действующих ТЭС."
568                      aria-label="Информация о коэффициенте ПГ">
569                <i class="bi bi-info-circle" aria-hidden="true"></i>
570              </button>
571              {% endif %}
572            </th>
573            {% endfor %}
574          </tr>
575          <tr class="text-center align-middle">
576            {% set char_symbols = {
577              'ash': 'A<sup>p</sup>, %',
578              'ash_share': 'a<sub>ун</sub>',
579              'q_mech': 'q<sub>4</sub>, %',
580              'sulfur': 'S<sup>p</sup>, %',
581              'qh_kj': 'Q<sub>н</sub>, кДж/кг',
582              'qh_kcal': 'Q<sub>н</sub>, ккал/кг',
583              'bj': 'b<sub>j</sub>',
584              'k_co2': 'K<sub>CO<sub>2</sub></sub>',
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 881–906.

```
881      elif layout == "chars":
882          for spec in _characteristic_row_specs():
883              is_header = spec["kind"] == KIND_TOTAL
884              is_group = is_header and not DEFAULT_CHARACTERISTICS.get(spec["key"])
885              values_by_year = {}
886              raw_values = []
887              for year in years:
888                  chars = resolve_chars(spec["key"], year, char_overrides, territory_slug)
889                  values_by_year[year] = {
890                      field: "" if is_group else format_cell(chars.get(field), digits=digits)
891                      for field, _label in CHAR_COLUMNS
892                  }
893                  raw_values.extend(None if is_group else chars.get(field) for field, _label in
CHAR_COLUMNS)
894              rows.append({
895                  "key": spec["key"],
896                  "label": spec["label"],
897                  "kind": spec["kind"],
898                  "indent": spec.get("indent") or 0,
899                  "italic": bool(spec.get("indent")),
900                  "css_class": "table-info fw-bold" if not spec.get("indent") else (
901                      "fw-bold" if spec["indent"] == 1 or is_header else ""
902                  ),
903                  "editable_values": not is_group,
904                  "char_values": values_by_year,
905                  "is_empty": False if is_header else not any(value is not None for value in
raw_values),
906              })
```

5.3. Коэффициенты очистки и подавления

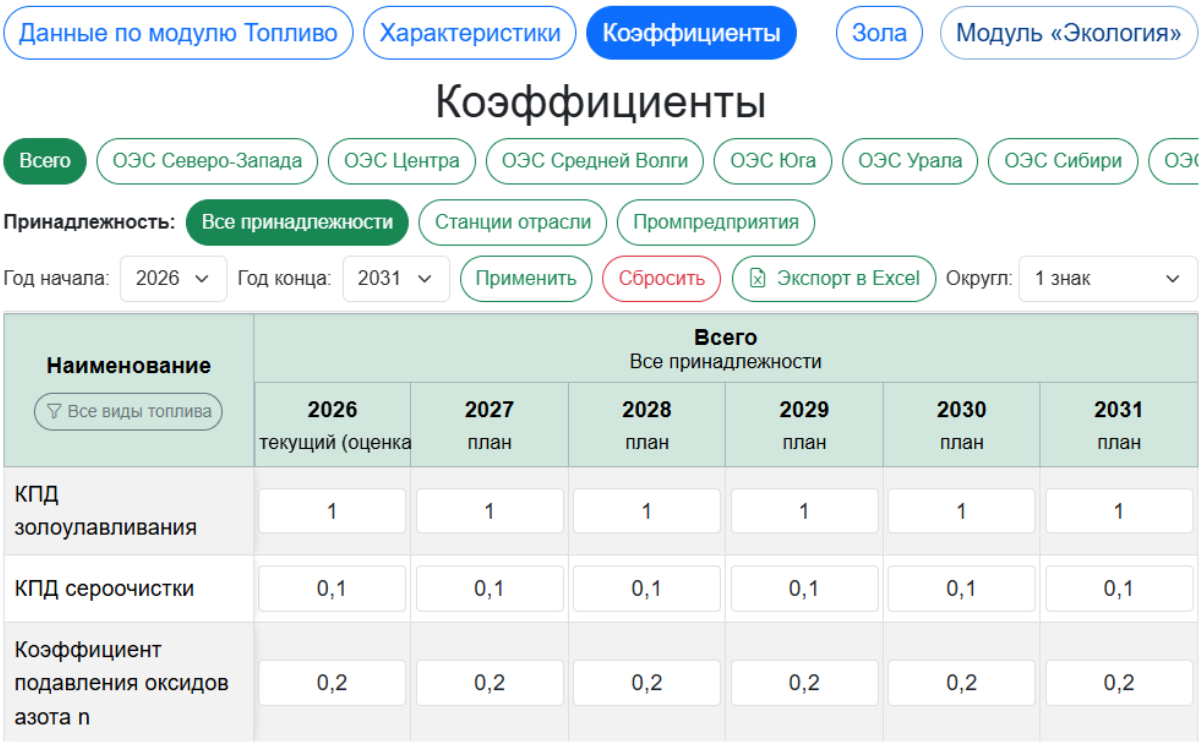
Адрес: /ecology/operating_tes/vsego/koeff/

Функциональные возможности и действия пользователя

Матрица коэффициентов содержит КПД золоулавливания и сероочистки, коэффициенты подавления NOx, CO и ПГ. Столбцы соответствуют годам выбранного периода. Территория и принадлежность выбираются независимо; доступны все принадлежности, станции отрасли и промпредприятия. При отсутствии сохраненного значения применяется предусмотренное кодом значение по умолчанию. Правка ячейки сохраняется через PATCH. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.



Наименование	Всего Все принадлежности					
Все виды топлива	2026 текущий (оценка	2027 план	2028 план	2029 план	2030 план	2031 план
КПД золоулавливания	1	1	1	1	1	1
КПД сероочистки	0,1	0,1	0,1	0,1	0,1	0,1
Коэффициент подавления оксидов азота n	0,2	0,2	0,2	0,2	0,2	0,2

Рисунок 10. Коэффициенты очистки и подавления: территория, принадлежность и годы. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_page_services.py, строки 1126–1148.

```
def upsert_coefficient(
    territory_slug: str, year: int, kind: str, value: Any, affiliation: str = "all"
) -> OperatingTesCoefficient:
    if get_territory(territory_slug) is None and get_coefficient_territory(territory_slug) is None:
        raise ValueError("Неизвестная территория")
    if affiliation not in {"all", "minen", "prompred"}:
        raise ValueError("Неизвестная принадлежность")
    if kind not in {item["key"] for item in COEFF_KINDS}:
        raise ValueError("Неизвестный коэффициент")
    # Подписи для журнала – до нормализации срезов принадлежности.
    log_context = context_lines(
        ("лист", "Коэффициенты"),
        ("территория", territory_label(territory_slug)),
        ("принадлежность", affiliation_label(affiliation)),
        ("год", int(year)),
    )
    log_action = f"Коэффициенты – {territory_label(territory_slug)}"
    # Сохраняем прежние общие срезы принадлежности, используемые на «Объемах».
    if territory_slug == "vsego" and affiliation != "all":
        territory_slug, affiliation = affiliation, "all"
    elif territory_slug in {"minen", "prompred"} and affiliation != "all":
        raise ValueError("Для принадлежности необходимо выбрать территорию")
    version_id = get_current_db_version_id()
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 797–820.

```
            {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
            <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
          data-editable-field="{{ row.editable_field or '' }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ row.label }}{% if row.key in ('ee_generation', 'heat_output') and row.unit %}, {{
row.unit }}{% endif %} {% if row.formula_tooltip %}
            <i
              class="bi bi-info-circle text-primary ms-1"
              data-bs-toggle="tooltip"
              data-bs-placement="top"
              {% if row.formula_tooltip_html %}data-bs-html="true" title="{{
row.formula_tooltip_html|forceescape }}"{% else %}title="{{ row.formula_tooltip|e }}"{% endif %}
              style="cursor: help; font-size: 0.9em"
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 798–811.

```
798     elif layout == "coeff":
799         coeff_overrides = load_coefficient_overrides(years, affiliation)
800         default_slug = (
801             affiliation if territory_slug == "vsego" and affiliation != "all" else territory_slug
802         )
803         for column in value_columns:
804             year = column["year"]
805             bundles_by_terr[column["slug"]] = {"coeffs": {
806                 kind["key"]: coeff_overrides.get(
807                     (territory_slug, year, kind["key"]),
808                     Decimal(str(default_coefficient(year, default_slug, kind["key"]))),
809                 )
810                 for kind in COEFF_KINDS
811             }}
```

5.4. Зола: выбросы твердых веществ

Адрес: /ecology/operating_tes/vsego/zola/

Функциональные возможности и действия пользователя

Для каждого вида топлива показаны перевод расхода в натуральное выражение по теплоте сгорания и масса отходящих твердых веществ. Формула учитывает долю уноса золы, зольность, механический недожог и теплоту сгорания. Итоговый блок показывает сумму Мотх., КПД улавливания и массу выброса Мвыб. Пользователь просматривает результат, а исходные свойства и КПД меняет на специальных вкладках. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 44–54.

```
def ash_motkh(bn: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх золы = 0,01·Вн·(аун·Ар + qч·QH/32680)."""
    bn_d = _d(bn)
    if bn_d is None or not chars:
        return None
    aun = _d(chars.get("ash_share")) or Decimal("0")
    ap = _d(chars.get("ash")) or Decimal("0")
    qch = _d(chars.get("q_mech")) or Decimal("0")
    qh = _d(chars.get("qh_kj")) or Decimal("0")
    inner = aun * ap + qch * qh / Q_H_DIVISOR
    return Decimal("0.01") * bn_d * inner
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ emission_row_label(row) }}
          </td>
          {% for col in value_columns %}
          {% if row.span_pair %}
          <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
            {% include 'ecology/operating_tes/_emission_summary_value.html' %}
          </td>
          {% else %}
          {% set pair =
          row.pair_values[col.slug] if row.pair_values is defined and col.slug
          in row.pair_values else none %}
          <td
            class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
          >
            {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

5.5. Сера: выбросы диоксида серы

Адрес: /ecology/operating_tes/vsego/sera/

Функциональные возможности и действия пользователя

Рассчитывается масса отходящего SO2 по натуральному расходу топлива, содержанию серы и доле ее связывания в золе. Натуральный расход для этого листа определяется по теплоте сгорания. Итоговая масса выброса учитывает КПД сероочистки. Расчетные ячейки доступны для просмотра, а не прямого ввода. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 57–64.

```
def so2_motkh(bn: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх SO2 = 0,02·Вн·Sp·(1 − x)."""
    bn_d = _d(bn)
    if bn_d is None or not chars:
        return None
    sp = _d(chars.get("sulfur")) or Decimal("0")
    x = _d(chars.get("sulfur_share")) or Decimal("0")
    return Decimal("0.02") * bn_d * sp * (Decimal("1") - x)
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
              {% endif %}
            </th>
            {% endfor %}
          </tr>
        </thead>
        <tbody>
          {% for row in rows %}
          <tr
            class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
            data-row-key="{{ row.key }}"
          >
            <td
              class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
            >
              {{ emission_row_label(row) }}
            </td>
            {% for col in value_columns %}
            {% if row.span_pair %}
            <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
              {% include 'ecology/operating_tes/_emission_summary_value.html' %}
            </td>
            {% else %}
            {% set pair =
            row.pair_values[col.slug] if row.pair_values is defined and col.slug
            in row.pair_values else none %}
            <td
              class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
            >
              {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

5.6. Азот: выбросы оксидов азота

Адрес: /ecology/operating_tes/vsego/azot/

Функциональные возможности и действия пользователя

Расчет Мотх.NOx использует расход условного топлива, механический недожог и коэффициент bj. В основном расчетном столбце показан результат без коэффициента подавления n; в итоговом блоке n применяется к суммарной массе, после чего формируется Мвыб. Такое разделение позволяет видеть исходную массу и эффект подавления. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 67–74.

```
def nox_motkh(b_tce: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх NOx без n: 0,001·B·(1 – qч/100)·bj."""
    b = _d(b_tce)
    if b is None or not chars:
        return None
    qch = _d(chars.get("q_mech")) or Decimal("0")
    bj = _d(chars.get("bj")) or Decimal("0")
    return Decimal("0.001") * b * (Decimal("1") - qch / Decimal("100")) * bj
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 727–753.

```
        <col class="ot-col-terr" />
        {% endfor %}
      </colgroup>
      <thead class="table-success">
        <tr class="text-center align-middle">
          <th class="text-center align-middle ot-label-col">
            Вид топлива
            {% include 'ecology/operating_tes/_fuel_filter.html' %}
            {% include 'ecology/operating_tes/_territory_filter.html' %}
          </th>
          {% for col in territory_columns %}
          <th class="text-center align-middle ot-terr-col">
            {{ col.sheet_name }}
            {% if emission_view.metric_formula_html %}
            <div class="fw-normal">M<sub>отх</sub>.
              <button type="button" class="btn btn-link p-0 text-primary ms-1"
                      data-bs-toggle="tooltip" data-bs-placement="top" data-bs-html="true"
                      data-bs-custom-class="ot-summary-tooltip"
                      title="{{ emission_view.metric_formula_html|forceescape }}" aria-
label="Формула Мотх {{ 'азота' if sheet.slug == 'azot' else 'ПГ' }}">
                <i class="bi bi-info-circle" aria-hidden="true"></i>
              </button>
            </div>
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

5.7. CO: выбросы монооксида углерода

Адрес: /ecology/operating_tes/vsego/co/

Функциональные возможности и действия пользователя

Рассчитывается масса CO по натуральному расходу топлива, химическому недожогу q3, коэффициенту R, теплоте сгорания и механическому недожогу q4. На этом листе масса CO представляется в тоннах, в отличие от ряда листов с тыс. т. Итоговый блок применяет коэффициент подавления CO. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 77–91.

```
def co_motkh(bn: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх CO, т: 0,001·ВН·(q3·R·Qн/1013)·(1 – q4/100)."""
    bn_d = _d(bn)
    if bn_d is None or not chars:
        return None
    q3 = _d(chars.get("q_chem")) or Decimal("0")
    r = _d(chars.get("r_chem")) or Decimal("0")
    qh = _d(chars.get("qh_kj")) or Decimal("0")
    qch = _d(chars.get("q_mech")) or Decimal("0")
    return (
        Decimal("0.001")
        * bn_d
        * (q3 * r * qh / CO_QH_DIVISOR)
        * (Decimal("1") - qch / Decimal("100"))
    )
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ emission_row_label(row) }}
          </td>
          {% for col in value_columns %}
          {% if row.span_pair %}
          <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
            {% include 'ecology/operating_tes/_emission_summary_value.html' %}
          </td>
          {% else %}
          {% set pair =
          row.pair_values[col.slug] if row.pair_values is defined and col.slug
          in row.pair_values else none %}
          <td
            class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
          >
            {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929     elif layout in {"emission", "emission_tj"}:
930         view = EMISSION_SHEET_VIEW[sheet["slug"]]
931         if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932             prefix, eta_key, eta_label = {
933                 "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                 "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                 "co": ("co", "co_k", "Коэффициент подавления CO"),
936                 "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                 "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                 "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939             }[sheet["slug"]]
940             total_key = f"{prefix}_motkh_total"
941             emitted_key = f"{prefix}_emitted"
942             summary_values = {}
943             for column in value_columns:
944                 slug = column["slug"]
945                 bundle = bundles_by_terr[slug]
946                 motkh = bundle[total_key]
947                 eta = bundle["coeffs"][eta_key]
948                 summary_values[slug] = {
949                     total_key: motkh,
950                     eta_key: eta,
951                     emitted_key: apply_capture(motkh, eta),
952                 }
953             for key, label in (
954                 (total_key, "ΣMотх."),
955                 (eta_key, eta_label),
956                 (emitted_key, "Mвыб."),
```

5.8. ПГ: прямой расчет CO2

Адрес: /ecology/operating_tes/vsego/pg/

Функциональные возможности и действия пользователя

Масса CO2 определяется как произведение расхода топлива и KCO2 по каждому виду топлива. Результат представлен в тыс. т; итоговый блок показывает массу до и после применения коэффициента подавления ПГ. Отсутствующий расход или коэффициент не подменяется произвольным расчетным числом. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 94–102.

```
def pg_motkh(b_tce: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх CO2 = B·KCO2, тыс. т."""
    b = _d(b_tce)
    if b is None or not chars:
        return None
    k = _d(chars.get("k_co2"))
    if k is None:
        return None
    return b * k
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 727–753.

```
        <col class="ot-col-terr" />
        {% endfor %}
      </colgroup>
      <thead class="table-success">
        <tr class="text-center align-middle">
          <th class="text-center align-middle ot-label-col">
            Вид топлива
            {% include 'ecology/operating_tes/_fuel_filter.html' %}
            {% include 'ecology/operating_tes/_territory_filter.html' %}
          </th>
          {% for col in territory_columns %}
          <th class="text-center align-middle ot-terr-col">
            {{ col.sheet_name }}
            {% if emission_view.metric_formula_html %}
            <div class="fw-normal">М<sub>отх</sub>.
              <button type="button" class="btn btn-link p-0 text-primary ms-1"
                      data-bs-toggle="tooltip" data-bs-placement="top" data-bs-html="true"
                      data-bs-custom-class="ot-summary-tooltip"
                      title="{{ emission_view.metric_formula_html|forceescape }}" aria-
label="Формула Мотх {{ 'азота' if sheet.slug == 'azot' else 'ПГ' }}">
                <i class="bi bi-info-circle" aria-hidden="true"></i>
              </button>
            </div>
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929     elif layout in {"emission", "emission_tj"}:
930         view = EMISSION_SHEET_VIEW[sheet["slug"]]
931         if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932             prefix, eta_key, eta_label = {
933                 "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                 "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                 "co": ("co", "co_k", "Коэффициент подавления CO"),
936                 "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                 "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                 "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939             }[sheet["slug"]]
940             total_key = f"{prefix}_motkh_total"
941             emitted_key = f"{prefix}_emitted"
942             summary_values = {}
943             for column in value_columns:
944                 slug = column["slug"]
945                 bundle = bundles_by_terr[slug]
946                 motkh = bundle[total_key]
947                 eta = bundle["coeffs"][eta_key]
948                 summary_values[slug] = {
949                     total_key: motkh,
950                     eta_key: eta,
951                     emitted_key: apply_capture(motkh, eta),
952                 }
953             for key, label in (
954                 (total_key, "ΣМотх."),
955                 (eta_key, eta_label),
956                 (emitted_key, "Мвыб."),
```

5.9. ПГ через ТДж: расчет CO2 по энергии топлива

Адрес: /ecology/operating_tes/vsego/pg-tj/

Функциональные возможности и действия пользователя

Альтернативный расчет переводит расход топлива в энергетическое выражение через коэффициенты т у.т./тн и ТДж/тыс. т, затем применяет коэффициент т CO2/ТДж. Энергия и масса показываются раздельно; масса CO2 выражена в тыс. т. В коде фактическая энергия рассчитывается в ТДж, хотя краткая подпись одного столбца макета содержит «МДж». Описание не приравнивает эти единицы. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 105–121.

```
def pg_tj_energy_and_mass(
    b_tce: Any, chars: Mapping[str, Any] | None
) -> tuple[Decimal | None, Decimal | None]:
    """Энергия в ТДж (столбец «МДж») и масса CO2 в тыс. т по коэффициентам НД."""
    b = _d(b_tce)
    if b is None or not chars:
        return None, None
    tut_per_t = _d(chars.get("tut_per_t"))
    tj_per_kt = _d(chars.get("tj_per_kt"))
    if tut_per_t is None or tut_per_t <= 0 or tj_per_kt is None:
        return None, None
    energy_tj = b / tut_per_t * tj_per_kt
    t_per_tj = _d(chars.get("t_co2_per_tj"))
    if t_per_tj is None:
        return energy_tj, None
    mass_thousand_t = energy_tj * t_per_tj / Decimal("1000")
    return energy_tj, mass_thousand_t
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
              {% endif %}
            </th>
            {% endfor %}
          </tr>
        </thead>
        <tbody>
          {% for row in rows %}
          <tr
            class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
            data-row-key="{{ row.key }}"
          >
            <td
              class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
            >
              {{ emission_row_label(row) }}
            </td>
            {% for col in value_columns %}
            {% if row.span_pair %}
            <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
              {% include 'ecology/operating_tes/_emission_summary_value.html' %}
            </td>
            {% else %}
            {% set pair =
            row.pair_values[col.slug] if row.pair_values is defined and col.slug
            in row.pair_values else none %}
            <td
              class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
            >
              {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

5.10. Объемы и удельные выбросы действующих ТЭС

Адрес: /ecology/operating_tes/vsego/obemy/

Функциональные возможности и действия пользователя

Сводная расчетная форма собирает массы золы, SO2, NOx, CO и CO2 по территориям после учета очистки и подавления. Дополнительные таблицы показывают удельные выбросы на приведенную энергию и на топливо. Используются согласованные единицы массы и энергии; CO отдельно учитывается в тоннах. Удельный показатель не вычисляется при нулевой базе деления. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

Взаимодействие и источники данных

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

Элементы реализации

Модель OperatingTesInputValue. Источник: app/ecology/models/operating_tes_input_model.py, строки 11–14.

```
11  class OperatingTesInputValue(db.Model, AuditMixin):
12      """Переопределение расхода топлива / выработки / тепла по территории и году."""
13
14      __tablename__ = "gs_eco_operating_tes_inputs"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_page_services.py, строки 652–670.

```
def _build_specific_emission_tables(
    value_columns: list[dict[str, Any]],
    bundles: dict[str, Any],
    inputs: dict[int, Any],
    digits: int,
) -> list[dict[str, Any]]:
    energy_title = "Удельные выбросы ЗВ и ПГ на единицу приведенной энергии, г/кВт.ч"
    electricity_title = "Удельные выбросы ЗВ и ПГ на единицу электрической энергии, г/кВт.ч"
    fuel_formula = "М × 1000 / В"
    solid_formula = "М × 100 / (Нефтетопливо + Торф + тв.топливо + Уголь всего)"
    brought_energy_formula = "М / (Выработка электроэнергии + Отпуск тепла / 859,8)"
    electricity_formula = "М / Выработка электроэнергии"
    tables = [
        {"key": "energy", "title": energy_title, "formula": brought_energy_formula},
        {"key": "energy_fuel", "title": electricity_title, "formula": electricity_formula},
        {"key": "fuel", "title": "Удельные выбросы ЗВ и ПГ на единицу топлива, кг/т у.т.",
         "formula": f"{fuel_formula}; для золы и SO2: {solid_formula}"},
    ]
    values = {table["key"]: {} for table in tables}
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 776–798.

```
      {% elif sheet.layout == 'volumes' %}
      {% include 'ecology/operating_tes/_volume_table_content.html' %}
      {% else %}
      <colgroup>
        <col class="ot-col-label" />
        {% for col in territory_columns %}
        <col class="ot-col-terr" />
        {% endfor %}
      </colgroup>
      <thead class="table-success">
        <tr class="text-center align-middle">
          <th class="text-center align-middle ot-label-col">Наименование
            {% if sheet.layout in ('fuel', 'coeff') %}
            {% include 'ecology/operating_tes/_fuel_filter.html' %}
            {% endif %}
            {% if territory_filter_storage_key %}
            {% include 'ecology/operating_tes/_territory_filter.html' %}
            {% endif %}
          </th>
          {% for col in territory_columns %}
          <th class="text-center align-middle ot-terr-col">
            {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
            <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 493–519.

```
def compute_year_bundle(
    territory_slug: str,
    year: int,
    bucket: dict[str, Decimal] | None,
    char_overrides: dict[tuple[str, str, int], dict[str, Any]],
    coeff_overrides: dict[tuple[str, int, str], Decimal],
) -> dict[str, Any]:
    coeffs = {
        kind["key"]: resolve_coeff(territory_slug, year, kind["key"], coeff_overrides)
        for kind in COEFF_KINDS
    }
    leaf_b: dict[str, Decimal | None] = {}
    by_key: dict[str, dict[str, Decimal | None]] = {}
    # leaves first
    for spec in FUEL_ROWS:
        if spec.get("is_header"):
            continue
        key = spec["key"]
        b_tce = fuel_amount_for_key(bucket, key)
        leaf_b[key] = b_tce
        chars = resolve_chars(key, year, char_overrides, territory_slug)
        by_key[key] = _compute_leaf_emissions(key, b_tce, chars, coeffs)
    header_specs = sorted(
        [spec for spec in FUEL_ROWS if spec.get("is_header")],
        key=lambda item: int(item.get("indent") or 0),
        reverse=True,
    )
```

6. Расчет по новым ТЭС

Для новых ТЭС нет самостоятельной формы «Добавить станцию»: состав поступает из топливной подсистемы, а редактируются только разрешенные нормативы. Передаваемые категории определены критериями сводки «Топлива». Внутренние ключи le100/gt100 исторические; актуальная передача new_lt100/new_ge100 разделяет заполненную мощность блока n1 на менее 100 МВт и 100 МВт или более. Старый низкоуровневый помощник _capacity_class_expr использует другой порог по Nуст; эти пути не следует смешивать при описании принятых снимков.

Категории передачи и их сопоставление с capacity_class. Источник: app/ecology/services/fuel_summary_constants.py, строки 3–14.

```
    3  FUEL_SUMMARY_SCOPES = (
    4      ("operating", "Действующие ТЭС"),
    5      ("new_lt100", "до 100 МВт"),
    6      ("new_ge100", "100 МВт и более"),
    7  )
    8  FUEL_SUMMARY_LABELS = dict(FUEL_SUMMARY_SCOPES)
    9  FUEL_SUMMARY_CRITERIA = {
   10      "operating": "Действующие группы оборудования действующих ТЭС с заполненной установленной
мощностью за выбранный год. Ограничений по величине мощности нет.",
   11      "new_lt100": "Новые группы оборудования действующих ТЭС или группы оборудования новых ТЭС.
Мощность блока (вар. 1) (n1) группы оборудования должна быть заполнена и составлять менее 100 МВт.",
   12      "new_ge100": "Новые группы оборудования действующих ТЭС или группы оборудования новых ТЭС.
Мощность блока (вар. 1) (n1) группы оборудования должна быть заполнена и составлять 100 МВт или более.",
   13  }
   14  NEW_TES_SCOPE_CAPACITY = {"new_lt100": "le100", "new_ge100": "gt100"}
```

6.1. Принятые топливные показатели новых ТЭС

Адрес: /ecology/new_tes/vsego/toplivo/

Функциональные возможности и действия пользователя

Показываются принятые показатели новых ТЭС для выбранного класса мощности: выработка, тепло и расход по видам топлива. Период и год выбираются в той же форме, что и для действующих ТЭС. Наличие нерассмотренного пакета показывается отдельно; отсутствие новых данных не означает отсутствия самой расчетной формы. Прямой ввод расхода на этом листе не разрешен.

Взаимодействие и источники данных

Используются два раздельных принятых снимка «Топлива» для новых ТЭС. Ключи capacity_class=le100 и gt100 сохраняются в навигации и сопоставлены с категориями пакетов new_lt100 и new_ge100. По критериям сводки передача разделяется по заполненной мощности блока n1: менее 100 МВт и 100 МВт или более. Обозначения le100/gt100 здесь являются внутренними ключами и не заменяют критерии отбора передаваемого пакета. Результаты обеих категорий включаются в сводные расчеты.

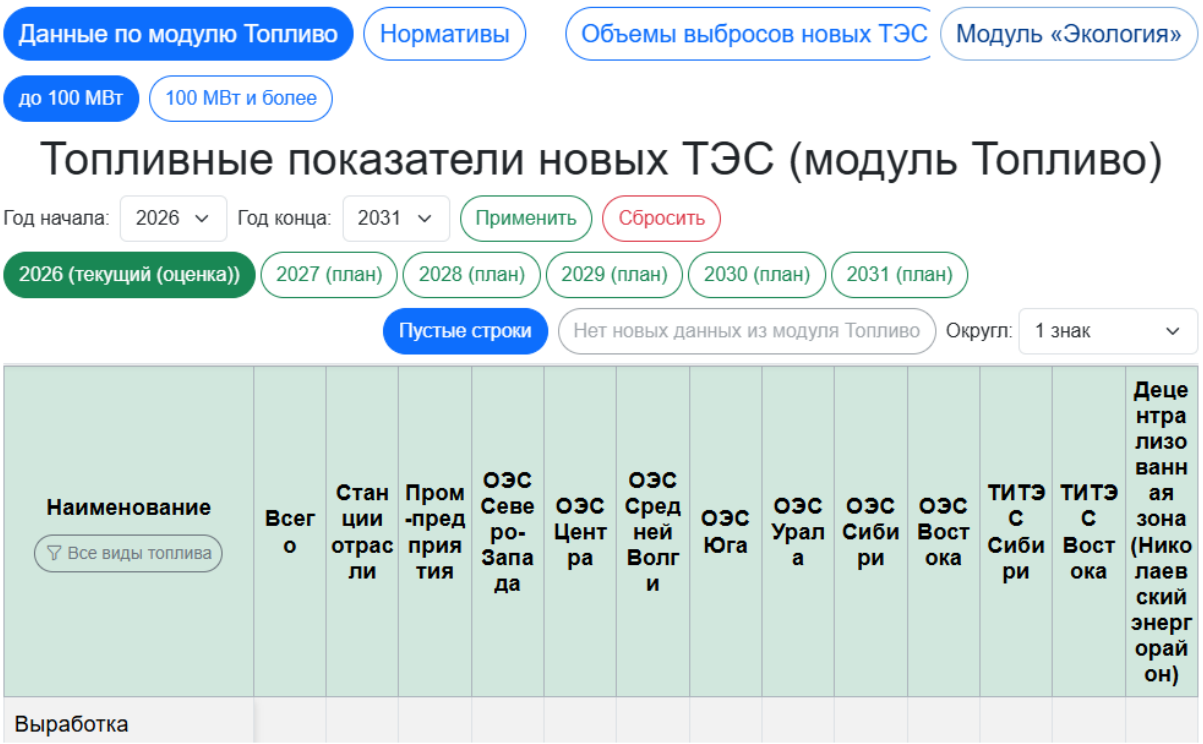


Рисунок 11. Новые ТЭС: принятые топливные показатели и выбор категории мощности. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
  11  class NewTesNorm(db.Model, AuditMixin):
  12      """Удельный выброс по виду топлива, классу мощности и году."""
  13
  14      __tablename__ = "gs_eco_new_tes_norms"
```

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
   9  class EcologyFuelSnapshot(db.Model, AuditMixin):
  10      __tablename__ = "gs_eco_fuel_snapshots"
  11      __table_args__ = (
  12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
  13          {"schema": SCHEMA_ECOLOGY},
  14      )
  15
  16      id = db.Column(db.Integer, primary_key=True)
  17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
  18      year_number = db.Column(db.Integer, nullable=False)
  19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
  20      payload = db.Column(db.JSON, nullable=False)
  21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/new_tes_routes.py, строки 58–72.

```
@ecology_bp.route("/new_tes/<territory_slug>/<sheet_slug>/")
@login_required
def new_tes_table(territory_slug: str, sheet_slug: str):
    context = build_new_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        view_year=request.args.get("view_year"),
        capacity_class=request.args.get("capacity_class", CAPACITY_LE100_KEY),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/fuel_exchange_services.py, строки 44–50.

```
def load_accepted_new_tes_fuel_inputs(years):
    result = {year: {} for year in years}
    for scope, capacity_class in NEW_TES_SCOPE_CAPACITY.items():
        for year, territories in load_accepted_fuel_inputs(years, scope=scope).items():
            for slug, bucket in territories.items():
                result[year].setdefault(slug, {})[capacity_class] = bucket
    return result
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 66–82.

```
            {% include 'ecology/operating_tes/_sheet_nav.html' %}
            {% if new_tes_capacity_tabs and not new_tes_norm_tables %}
            <nav class="d-flex flex-wrap gap-2" aria-label="Категории новых ТЭС">
              {% for capacity_key, capacity_label in new_tes_capacity_tabs %}
              <a class="btn {{ 'btn-primary' if capacity_class == capacity_key else 'btn-outline-
primary' }}"
                 {% if capacity_class == capacity_key %}aria-current="page"{% endif %}
                 href="{{ url_for(table_endpoint, territory_slug=territory.slug,
sheet_slug=sheet.slug, capacity_class=capacity_key, start_year=start_year, end_year=end_year,
view_year=view_year, rounding_digits=rounding_digits) }}">{{ capacity_label }}</a>
              {% endfor %}
            </nav>
            {% endif %}
            {% endif %}
          </div>
        </div>
      </div>

      {% if fuel_summary_errors %}
      <div class="alert alert-warning" role="alert">
```

Сервис новых ТЭС. Источник: app/ecology/services/new_tes_page_services.py, строки 276–291.

```
    if layout == "fuel":
        raw = load_accepted_new_tes_fuel_inputs(years)
        inputs = {
            year: {
                slug: by_class.get(capacity_class) or empty_fuel_bucket()
                for slug, by_class in (by_terr or {}).items()
            }
            for year, by_terr in raw.items()
        }
    elif layout == "chars":
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
    else:
        raw = load_accepted_new_tes_fuel_inputs(years)
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
```

6.2. Нормативы удельных выбросов новых ТЭС

Адрес: /ecology/new_tes/vsego/spravochnik/

Функциональные возможности и действия пользователя

Справочная форма содержит нормативы золы, SO2 и NOx в кг/т у.т. и CO2 в т/т у.т. Таблицы нормативов формируются для обеих категорий мощности. Разрешенные исходные ячейки сохраняются по виду топлива, классу и году. Эффективный коэффициент CO2 берется из общих характеристик топлива; его изменение на этой странице передается в общий k_co2, а не образует независимую копию норматива ПГ. Норматив CO в текущем расчете новых ТЭС принимается равным нулю.

Взаимодействие и источники данных

Используются два раздельных принятых снимка «Топлива» для новых ТЭС. Ключи capacity_class=le100 и gt100 сохраняются в навигации и сопоставлены с категориями пакетов new_lt100 и new_ge100. По критериям сводки передача разделяется по заполненной мощности блока n1: менее 100 МВт и 100 МВт или более. Обозначения le100/gt100 здесь являются внутренними ключами и не заменяют критерии отбора передаваемого пакета. Результаты обеих категорий включаются в сводные расчеты.

Элементы реализации

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
  11  class NewTesNorm(db.Model, AuditMixin):
  12      """Удельный выброс по виду топлива, классу мощности и году."""
  13
  14      __tablename__ = "gs_eco_new_tes_norms"
```

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
   9  class EcologyFuelSnapshot(db.Model, AuditMixin):
  10      __tablename__ = "gs_eco_fuel_snapshots"
  11      __table_args__ = (
  12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
  13          {"schema": SCHEMA_ECOLOGY},
  14      )
  15
  16      id = db.Column(db.Integer, primary_key=True)
  17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
  18      year_number = db.Column(db.Integer, nullable=False)
  19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
  20      payload = db.Column(db.JSON, nullable=False)
  21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/new_tes_routes.py, строки 58–72.

```
  58  @ecology_bp.route("/new_tes/<territory_slug>/<sheet_slug>/")
  59  @login_required
  60  def new_tes_table(territory_slug: str, sheet_slug: str):
  61      context = build_new_tes_table_context(
  62          territory_slug,
  63          sheet_slug,
  64          start_year=request.args.get("start_year"),
  65          end_year=request.args.get("end_year"),
  66          rounding_digits=request.args.get("rounding_digits"),
  67          view_year=request.args.get("view_year"),
  68          capacity_class=request.args.get("capacity_class", CAPACITY_LE100_KEY),
  69      )
  70      if context is None:
  71          abort(404)
  72      sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/new_tes_page_services.py, строки 137–155.

```
 137  def resolve_norm(
 138      fuel_key: str,
 139      capacity_class: str,
 140      year: int,
 141      overrides: dict[tuple[str, str, int], dict[str, Any]],
 142      char_overrides: dict | None = None,
 143  ) -> dict[str, Decimal]:
 144      out: dict[str, Decimal] = {}
 145      extra = overrides.get((fuel_key, capacity_class, year)) or {}
 146      for kind in NORM_KINDS:
 147          raw = extra.get(kind)
 148          if raw is None:
 149              raw = default_norm(fuel_key, capacity_class, kind)
 150          amount = _d(raw)
 151          out[kind] = amount if amount is not None else Decimal("0")
 152      out["pg"] = _d(resolve_chars(fuel_key, year, char_overrides or {}).get("k_co2")) or
Decimal("0")
 153      # Норматив CO не задан в новой таблице нормативов.
 154      out["co"] = Decimal("0")
 155      return out
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 524–548.

```
 524      {% for norm_table in new_tes_norm_tables|default([]) or [{'rows': rows, 'capacity_class':
capacity_class|default(none)}] %}
 525      {% with rows=norm_table.rows, capacity_class=norm_table.capacity_class %}
 526      {% if norm_table.title %}
 527      <section aria-labelledby="ot-norm-{{ capacity_class }}-title">
 528      <h2 class="h3 fw-bold mt-4 mb-3" id="ot-norm-{{ capacity_class }}-title">{{ norm_table.title
}}</h2>
 529      {% endif %}
 530      <table
 531        class="table table-hover table-striped table-bordered align-middle power-demand-summary-table
ot-calculation-table{% if new_tes_volume_columns %} ot-new-tes-volumes-table{% endif %}{% if sheet.layout
not in ('fuel', 'coeff') %} ot-hide-empty{% endif %}{% if sheet.layout in ('fuel', 'coeff', 'volumes') %}
ot-fit-table{% elif sheet.layout == 'chars' %} ot-chars-table{% elif emission_view and emission_view.kind
== 'pair' %} ot-emission-pair-table{% elif emission_view %} ot-emission-single-table{% endif %}"
 532        id="{{ 'operatingTesTable' if loop.first else 'operatingTesTable-' ~ capacity_class }}"
 533        {% if norm_table.title %}aria-labelledby="ot-norm-{{ capacity_class }}-title"{% endif %}
 534        {% if new_tes_volume_columns %}style="min-width: {{ 15 + value_columns|length * 8.5 }}rem;"{%
endif %}
 535        data-update-url="{{ url_for(value_endpoint, territory_slug=territory.slug,
sheet_slug=sheet.slug, capacity_class=capacity_class|default(none)) }}"
 536        data-can-edit="{{ 1 if can_edit else 0 }}"
 537        data-affiliation="{{ affiliation|default('all') }}"
 538        data-autosave="{{ 1 if sheet.layout in ('coeff', 'chars', 'emission', 'emission_tj') or
year_columns_mode else 0 }}"
 539        data-rounding-digits="{{ rounding_digits }}"
 540        data-coefficient-rounding-digits="{{ coefficient_rounding_digits|default(0) }}"
 541      >
 542        {% if new_tes_volume_columns %}
 543        {% include 'ecology/operating_tes/_new_tes_volume_table.html' %}
 544        {% elif dynamics_territories %}
 545        {% include 'ecology/operating_tes/_volume_table_content.html' %}
 546        {% elif year_columns_mode %}
 547        {% include 'ecology/operating_tes/_range_table.html' %}
 548        {% elif sheet.layout == 'chars' %}
```

Сервис новых ТЭС. Источник: app/ecology/services/new_tes_page_services.py, строки 478–498.

```
478  def upsert_norm(fuel_key: str, year: int, field: str, value: Any) -> NewTesNorm:
479      parsed = parse_norm_field(field)
480      if parsed is None or field not in NORM_FIELD_KEYS:
481          raise ValueError("Неизвестное поле норматива")
482      kind, capacity_class = parsed
483      if kind == "pg":
484          return upsert_characteristic(DEFAULT_TERRITORY_SLUG, fuel_key, year, "k_co2", value)
485      if fuel_key not in DEFAULT_NORMS:
486          raise ValueError("Неизвестный вид топлива")
487      version_id = get_current_db_version_id()
488      log_context = context_lines(
489          ("лист", "Нормативы"),
490          ("топливо", fuel_label(fuel_key)),
491          ("год", int(year)),
492      )
493      query = NewTesNorm.query.filter_by(
494          fuel_key=fuel_key,
495          capacity_class=capacity_class,
496          year_number=int(year),
497      )
498      if version_id is not None:
```

6.3. Объемы выбросов новых ТЭС

Адрес: /ecology/new_tes/vsego/obemy/

Функциональные возможности и действия пользователя

Для выбранной категории мощности рассчитываются ожидаемые массы загрязняющих веществ и CO2 по видам топлива и территориям. Базовая формула М = В × У. При расходе в тыс. т у.т. и нормативе в кг/т у.т. результат ЗВ получается в тоннах; для CO2 норматив в т/т у.т. дает тыс. т. В суммарных формах единицы приводятся явно. Расчетные ячейки не редактируются напрямую.

Взаимодействие и источники данных

Используются два раздельных принятых снимка «Топлива» для новых ТЭС. Ключи capacity_class=le100 и gt100 сохраняются в навигации и сопоставлены с категориями пакетов new_lt100 и new_ge100. По критериям сводки передача разделяется по заполненной мощности блока n1: менее 100 МВт и 100 МВт или более. Обозначения le100/gt100 здесь являются внутренними ключами и не заменяют критерии отбора передаваемого пакета. Результаты обеих категорий включаются в сводные расчеты.

Элементы реализации

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/new_tes_routes.py, строки 58–72.

```
@ecology_bp.route("/new_tes/<territory_slug>/<sheet_slug>/")
@login_required
def new_tes_table(territory_slug: str, sheet_slug: str):
    context = build_new_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        view_year=request.args.get("view_year"),
        capacity_class=request.args.get("capacity_class", CAPACITY_LE100_KEY),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/new_tes_calc_services.py, строки 12–18.

```
def zv_mass_tonnes(b_tce: Any, u_kg_per_tce: Any) -> Decimal | None:
    """М, т = В, тыс. т у.т. × У, кг/т у.т."""
    b = _d(b_tce)
    u = _d(u_kg_per_tce)
    if b is None or u is None:
        return None
    return b * u
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 539–548.

```
      data-rounding-digits="{{ rounding_digits }}"
      data-coefficient-rounding-digits="{{ coefficient_rounding_digits|default(0) }}"
    >
      {% if new_tes_volume_columns %}
      {% include 'ecology/operating_tes/_new_tes_volume_table.html' %}
      {% elif dynamics_territories %}
      {% include 'ecology/operating_tes/_volume_table_content.html' %}
      {% elif year_columns_mode %}
      {% include 'ecology/operating_tes/_range_table.html' %}
      {% elif sheet.layout == 'chars' %}
```

Сервис новых ТЭС. Источник: app/ecology/services/new_tes_page_services.py, строки 193–218.

```
def compute_new_tes_year_bundle(
    territory_slug: str,
    year: int,
    by_class: dict[str, dict[str, Decimal]] | None,
    norm_overrides: dict[tuple[str, str, int], dict[str, Any]],
    char_overrides: dict | None = None,
) -> dict[str, Any]:
    by_key: dict[str, dict[str, Decimal | None]] = {}
    for spec in FUEL_ROWS:
        if spec.get("is_header"):
            continue
        by_key[spec["key"]] = _leaf_emissions(
            spec["key"], by_class, year, norm_overrides, char_overrides
        )
    header_specs = sorted(
        [spec for spec in FUEL_ROWS if spec.get("is_header")],
        key=lambda item: int(item.get("indent") or 0),
        reverse=True,
    )
    for spec in header_specs:
        children = tuple(spec.get("children") or PARENT_CHILDREN.get(spec["key"]) or ())
        computed = {metric: None for metric in _BUNDLE_METRICS}
        for metric in _BUNDLE_METRICS:
            computed[metric] = _sum_present(
                [(by_key.get(child) or {}).get(metric) for child in children]
            )
```

7. Сводные данные расчетов

7.1. Общие топливные показатели действующих и новых ТЭС

Адрес: /ecology/calculation_summary/vsego/toplivo/

Функциональные возможности и действия пользователя

Суммируются принятые топливные и энергетические показатели действующих ТЭС и двух категорий новых ТЭС. Строки и территориальные столбцы сопоставляются по внутренним ключам, а не по положению в таблице. Форма позволяет выбирать период и год и не разрешает прямую правку суммарных ячеек. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

Взаимодействие и источники данных

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

Элементы реализации

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 32–41.

```
  def _sum_rows(context, sources):
      source_rows = [
          {row["key"]: row for row in source["rows"]} for source in sources
      ]
      source_columns = [
          {col.get("territory_slug", col["slug"]): col["slug"] for col in source["value_columns"]}
          for source in sources
      ]
      territories = {col["slug"]: col.get("territory_slug", col["slug"]) for col in
context["value_columns"]}
      rows = []
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 787–820.

```
            <th class="text-center align-middle ot-label-col">Наименование
              {% if sheet.layout in ('fuel', 'coeff') %}
              {% include 'ecology/operating_tes/_fuel_filter.html' %}
              {% endif %}
              {% if territory_filter_storage_key %}
              {% include 'ecology/operating_tes/_territory_filter.html' %}
              {% endif %}
            </th>
            {% for col in territory_columns %}
            <th class="text-center align-middle ot-terr-col">
              {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
              <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
              {% endif %}
            </th>
            {% endfor %}
          </tr>
        </thead>
        <tbody>
          {% for row in rows %}
          <tr
            class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
            data-row-key="{{ row.key }}"
            data-editable-field="{{ row.editable_field or '' }}"
          >
            <td
              class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
            >
              {{ row.label }}{% if row.key in ('ee_generation', 'heat_output') and row.unit %}, {{
row.unit }}{% endif %} {% if row.formula_tooltip %}
              <i
                class="bi bi-info-circle text-primary ms-1"
                data-bs-toggle="tooltip"
                data-bs-placement="top"
                {% if row.formula_tooltip_html %}data-bs-html="true" title="{{
row.formula_tooltip_html|forceescape }}"{% else %}title="{{ row.formula_tooltip|e }}"{% endif %}
                style="cursor: help; font-size: 0.9em"
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 185–196.

```
            return None
        if sheet_slug == "toplivo":
            context = build_operating_tes_table_context(territory_slug, source_sheet, **filters)
            context["rows"] = _sum_rows(context, [context, le100, gt100])
            context["page_title"] = "Топливные показатели действующих и новых ТЭС (модуль Топливо)"
            context["page_title_html"] = context["page_title"]
        else:
            new_sources = [_transpose_new_volumes(source) for source in (le100, gt100)]
            context = {**new_sources[0], "rows": _sum_rows(new_sources[0], new_sources)}
            if sheet_slug in {"russia", "specific"}:
                operating = build_operating_tes_table_context(territory_slug, source_sheet,
**filters)
                context = {**operating, "rows": _sum_rows(operating, [operating, context])}
```

7.2. Объемы выбросов по России

Адрес: /ecology/calculation_summary/vsego/russia/

Функциональные возможности и действия пользователя

Выводятся общий объем и отдельные блоки действующих ТЭС, новых ТЭС le100 и новых ТЭС gt100. Перед сложением массы ЗВ новых ТЭС переводятся из тонн в тыс. т там, где этого требует общая форма; отдельная строка CO остается в тоннах, CO2 в тыс. т. Расчет использует принятые данные, а не сумму видимых округленных строк браузера. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

Взаимодействие и источники данных

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

Элементы реализации

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 59–79.

```
 59  def _transpose_new_volumes(context):
 60      total = context["rows"][0]
 61      columns = context["territory_columns"]
 62      rows = []
 63      for spec in VOLUME_ROWS:
 64          metric = spec["key"]
 65
 66          def amount(slug):
 67              value = decimal_or_none(total["col_values_raw"][f"{slug}:{metric}"])
 68              # В исходной таблице новых ТЭС ЗВ – в тоннах. Форма действующих
 69              # ТЭС использует тыс. т, кроме отдельной строки CO (тонны).
 70              if value is not None and metric not in {"co", "co2"}:
 71                  value /= Decimal("1000")
 72              return value
 73
 74          values, raw = _cells_for_territories(columns, amount, context["rounding_digits"])
 75          rows.append(_row_dict(
 76              key=metric, label=spec["label"], unit="т" if metric == "co" else "тыс. т",
 77              kind=spec["kind"], indent=spec["indent"],
 78              col_values=values, col_values_raw=raw,
 79          ))
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 853–862.

```
 853      {% for volume_table in summary_volume_tables %}
 854      <h2 class="h3 fw-bold mt-4 mb-3" id="ot-summary-{{ volume_table.key }}-title">
 855        {{ volume_table.title }}
 856      </h2>
 857      <table class="table table-hover table-striped table-bordered align-middle power-demand-summary-
table ot-calculation-table ot-fit-table ot-hide-empty"
 858             id="ot-summary-{{ volume_table.key }}" aria-labelledby="ot-summary-{{ volume_table.key
}}-title" data-can-edit="0">
 859        {% with rows = volume_table.rows %}
 860        {% include 'ecology/operating_tes/_volume_table_content.html' %}
 861        {% endwith %}
 862      </table>
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 191–206.

```
 191          else:
 192              new_sources = [_transpose_new_volumes(source) for source in (le100, gt100)]
 193              context = {**new_sources[0], "rows": _sum_rows(new_sources[0], new_sources)}
 194              if sheet_slug in {"russia", "specific"}:
 195                  operating = build_operating_tes_table_context(territory_slug, source_sheet,
**filters)
 196                  context = {**operating, "rows": _sum_rows(operating, [operating, context])}
 197                  context["specific_emission_tables"] = []
 198                  if sheet_slug == "russia":
 199                      context["main_table_title"] = "Всего"
 200                      context["summary_volume_tables"] = [
 201                          {"key": "operating", "title": "Действующие ТЭС", "rows":
operating["rows"]},
 202                          {"key": "new-gt100", "title": "Новые ТЭС – 100 МВт и более",
 203                           "rows": _sum_rows(operating, [new_sources[1]])},
 204                          {"key": "new-le100", "title": "Новые ТЭС – менее 100 МВт",
 205                           "rows": _sum_rows(operating, [new_sources[0]])},
 206                      ]
```

7.3. Удельные выбросы по России

Адрес: /ecology/calculation_summary/vsego/specific/

Функциональные возможности и действия пользователя

Из общего объема выбросов и общей топливно-энергетической базы формируются удельные показатели всех ТЭС. Сначала суммируются массы и соответствующие базы, затем вычисляются удельные значения. Простое сложение или среднее арифметическое удельных выбросов отдельных категорий не используется. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

Взаимодействие и источники данных

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

Элементы реализации

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 86–105.

```
 86  def _build_russia_specific_emission_tables(context, fuel_context):
 87      inputs = {}
 88      bundles = {}
 89      for column in context["value_columns"]:
 90          slug = column["slug"]
 91          bucket = {}
 92          for row in fuel_context["rows"]:
 93              amount = decimal_or_none(row["col_values_raw"][slug])
 94              if row["key"] in TOP_INPUT_FIELD:
 95                  apply_input_override_to_bucket(bucket, row["key"], amount)
 96              elif amount is not None:
 97                  bucket[row["key"]] = amount
 98          inputs.setdefault(column["year"], {})[column["territory_slug"]] = bucket
 99          bundles[slug] = {"volumes": {
100              row["key"]: decimal_or_none(row["col_values_raw"][slug])
101              for row in context["rows"]
102          }}
103      return _build_specific_emission_tables(
104          context["value_columns"], bundles, inputs, context["rounding_digits"],
105      )
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 863–879.

```
863      {% endfor %}
864      {% for specific_table in specific_emission_tables %}
865      <div class="d-flex align-items-center justify-content-between gap-2 mt-4 mb-3">
866        <h2 class="h3 fw-bold mb-0 flex-grow-1" id="ot-specific-{{ specific_table.key }}-title">
867          {{ specific_table.title }}
868        </h2>
869        {% if specific_table.key == 'energy' %}
870        {{ rounding_control('specific_rounding_digits', 'Округл (удельные)',
specific_rounding_digits) }}
871        {% endif %}
872      </div>
873      <table class="table table-hover table-striped table-bordered align-middle power-demand-summary-
table ot-calculation-table ot-fit-table ot-specific-table ot-hide-empty"
874             id="ot-specific-{{ specific_table.key }}" aria-labelledby="ot-specific-{{
specific_table.key }}-title" data-can-edit="0">
875        {% with rows = specific_table.rows %}
876        {% include 'ecology/operating_tes/_volume_table_content.html' %}
877        {% endwith %}
878      </table>
879      {% endfor %}
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 206–212.

```
206                      ]
207                  else:
208                      fuel_context = build_calculation_summary_context(territory_slug, "toplivo",
**filters)
209                      context["specific_emission_tables"] =
_build_russia_specific_emission_tables(context, fuel_context)
210                      context["specific_emissions_only"] = True
211                      context["page_title"] = "Удельные выбросы по России (все ТЭС)"
212                      context["page_title_html"] = context["page_title"]
```

7.4. Динамика изменения выбросов

Адрес: /ecology/calculation_summary/vsego/dynamics/

Функциональные возможности и действия пользователя

Сравниваются показатели по годам выбранного периода. Доступны территориальные вкладки динамики; строки формируются на основе общих расчетов действующих и новых ТЭС. Форма предназначена для просмотра временного изменения показателей и не создает отдельного набора введенных выбросов. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

Взаимодействие и источники данных

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

Элементы реализации

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 108–120.

```
108  def _build_emission_dynamics(territory_slug, filters):
109      context = build_calculation_summary_context(territory_slug, "russia", **filters)
110      annual = {
111          year: context if year == context["view_year"] else build_calculation_summary_context(
112              territory_slug, "russia", **{**filters, "view_year": year},
113          )
114          for year in context["range_years"]
115      }
116      columns = [
117          {"slug": f"{territory_slug}:{year}", "territory_slug": territory_slug,
118           "year": year, "sheet_name": str(year)}
119          for year in annual
120      ]
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/_sheet_nav.html, строки 34–42.

```
  {% if dynamics_territories %}
  <div class="d-flex flex-wrap gap-2" role="navigation" aria-label="Территории динамики выбросов">
    {% for nav in dynamics_territories %}
    <a href="{{ url_for(table_endpoint, territory_slug=nav.slug, sheet_slug=sheet_slug,
start_year=start_year, end_year=end_year, rounding_digits=rounding_digits,
coefficient_rounding_digits=coefficient_rounding_digits|default(none)) }}"
       class="btn {% if nav.slug == territory_slug %}btn-primary{% else %}btn-outline-primary{% endif
%} text-nowrap"{% if nav.slug == territory_slug %} aria-current="page"{% endif %}>
      {{ nav.sheet_name }}
    </a>
    {% endfor %}
  </div>
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 108–134.

```
def _build_emission_dynamics(territory_slug, filters):
    context = build_calculation_summary_context(territory_slug, "russia", **filters)
    annual = {
        year: context if year == context["view_year"] else build_calculation_summary_context(
            territory_slug, "russia", **{**filters, "view_year": year},
        )
        for year in context["range_years"]
    }
    columns = [
        {"slug": f"{territory_slug}:{year}", "territory_slug": territory_slug,
         "year": year, "sheet_name": str(year)}
        for year in annual
    ]

    def range_rows(tables):
        rows_by_year = {
            year: {row["key"]: row for row in table["rows"]}
            for year, table in tables.items()
        }
        rows = []
        for row in context["rows"]:
            amounts = {
                column["slug"]: decimal_or_none(
                    rows_by_year[column["year"]][row["key"]]["col_values_raw"][column["slug"]]
                )
                for column in columns
            }
```

8. Агрегированные суммы отчетно-аналитического контура

После группировки исходные абсолютные показатели суммируются, а производные коэффициенты и удельные показатели вычисляются заново на суммарной базе. Складывать готовые удельные значения станций нельзя. Представления с котельными и без котельных имеют разные исходные выборки.

Суммирование с повторным вычислением производных показателей. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3014–3035.

```
3014  def _sum_filled_rows(
3015      filled_list: list[list[Any]],
3016      source_columns: list[dict[str, Any]],
3017      norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
3018  ) -> list[Any] | None:
3019      if not filled_list:
3020          return None
3021      width = max(len(source_columns), max((len(row) for row in filled_list), default=0))
3022      out: list[Any] = [None] * width
3023      for index, column in enumerate(source_columns):
3024          key = str(column.get("key") or "")
3025          if key in SUMMARY_IDENTITY_KEYS or key in SUMMARY_RECOMPUTE_KEYS:
3026              continue
3027          if not column.get("numeric"):
3028              continue
3029          parts = [
3030              row[index] if index < len(row) else None
3031              for row in filled_list
3032          ]
3033          out[index] = _safe_sum(*parts)
3034      # Пересчет d_*/K_*/CO2/UD_* поверх сумм слагаемых (если столбцы есть).
3035      return _compute_derived_columns(out, source_columns, norm_zv_parts)
```

8.1. ПГ: агрегированные суммы с котельными

Адрес: /ecology/totals_summary/pg-vse/

Функциональные возможности и действия пользователя

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются исходные расходы и массы CO2; разности, коэффициенты эмиссии и удельные показатели формируются из суммарной базы. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

Взаимодействие и источники данных

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

Элементы реализации

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
 11  class PGParamCalc(db.Model, AuditMixin):
 12      """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.
 13
 14      Первоначально копируется из PGReportGisTek; далее может
 15      редактироваться отдельно от отчетных данных.
 16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
 17      (без FK на numb – поле не уникально).
 18      year_number + database_version_id → Year.number + Year.database_version_id.
 19      """
 20
 21      __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
@ecology_bp.route("/totals_summary/<sheet_slug>")
@ecology_bp.route("/totals_summary/<sheet_slug>/")
@login_required
def totals_summary(sheet_slug: str):
    context = _ecology_totals_page_context(sheet_slug)
    if context is None:
        abort(404)
    log_page_visit(
        "Итоговые суммы",
        tab=(context.get("sheet") or {}).get("page_title"),
        details=context_lines(
            ("лист", sheet_slug),
            ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
            ("округление", context.get("rounding_digits")),
        ),
    )
    return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
def build_region_summary_block(
    groups: list[dict[str, Any]],
    label: str,
    *,
    source_columns: list[dict[str, Any]],
    visible_columns: list[dict[str, Any]],
    digits: int,
    key_prefix: str,
    emission_digits: int | None = None,
    specific_digits: int | None = None,
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> dict[str, Any] | None:
    """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
    by_year: dict[int, list[list[Any]]] = defaultdict(list)
    for group in groups:
        for year, filled in (group.get("filled_by_year") or {}).items():
            if year is None or not filled:
                continue
            by_year[int(year)].append(list(filled))
    if not by_year:
        return None
    summary_label = f"{label}, всего"
    rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник:
app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
  91        <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}"
id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
  92          <input type="hidden" name="start_year" value="{{ start_year }}">
  93          <input type="hidden" name="end_year" value="{{ end_year }}">
  94
  95          <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type"
value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
  96          <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>
  97
  98          <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type"
value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
  99          <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>
 100
 101          <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type"
value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
 102          <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-
primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width:
80px;">Россия</label>
 103
 104          <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type"
value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
 105          <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>
 106
 107          <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type"
value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
 108          <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>
 109
 110          {% if sorted_synchronous_area_ids %}
 111            {% for sa_id in sorted_synchronous_area_ids %}
 112              {% set sa_name = synchronous_area_names[sa_id] %}
 113              {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
 114              {% set sa_value = 'sync_area_' + sa_id|string %}
 115              {% set is_checked = sa_value in aggregation_types %}
 116              <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}"
name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
 117              <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}"
style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
 118            {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
 305      should_show_totals = {
 306          "energy_system_types": {},
 307          "union_energy_systems": False,
 308          "synchronous_areas": {},
 309          "federal_districts": False,
 310          "total": False,
 311      }
 312      if "russia" in agg_types:
 313          should_show_totals["total"] = True
 314      if "fo" in agg_types:
 315          should_show_totals["federal_districts"] = True
 316      if "oes" in agg_types:
 317          should_show_totals["union_energy_systems"] = True
 318      for est_id in sorted_energy_system_type_ids:
 319          name_u = (energy_system_type_names.get(est_id) or "").upper()
 320          if "ees" in agg_types and "ЕЭС" in name_u:
 321              should_show_totals["energy_system_types"][est_id] = True
 322          if "tites" in agg_types and "ТИТЭС" in name_u:
 323              should_show_totals["energy_system_types"][est_id] = True
 324      for sa_id in sorted_synchronous_area_ids:
 325          if f"sync_area_{sa_id}" in agg_types:
 326              should_show_totals["synchronous_areas"][sa_id] = True
```

8.2. ПГ: агрегированные суммы без котельных

Адрес: /ecology/totals_summary/pg-bez-kot/

Функциональные возможности и действия пользователя

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются исходные расходы и массы CO2; разности, коэффициенты эмиссии и удельные показатели формируются из суммарной базы. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

Взаимодействие и источники данных

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

Элементы реализации

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
11  class PGParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.
13
14      Первоначально копируется из PGReportGisTek; далее может
15      редактироваться отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
64  @ecology_bp.route("/totals_summary/<sheet_slug>")
65  @ecology_bp.route("/totals_summary/<sheet_slug>/")
66  @login_required
67  def totals_summary(sheet_slug: str):
68      context = _ecology_totals_page_context(sheet_slug)
69      if context is None:
70          abort(404)
71      log_page_visit(
72          "Итоговые суммы",
73          tab=(context.get("sheet") or {}).get("page_title"),
74          details=context_lines(
75              ("лист", sheet_slug),
76              ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
77              ("округление", context.get("rounding_digits")),
78          ),
79      )
80      return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
def build_region_summary_block(
    groups: list[dict[str, Any]],
    label: str,
    *,
    source_columns: list[dict[str, Any]],
    visible_columns: list[dict[str, Any]],
    digits: int,
    key_prefix: str,
    emission_digits: int | None = None,
    specific_digits: int | None = None,
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> dict[str, Any] | None:
    """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
    by_year: dict[int, list[list[Any]]] = defaultdict(list)
    for group in groups:
        for year, filled in (group.get("filled_by_year") or {}).items():
            if year is None or not filled:
                continue
            by_year[int(year)].append(list(filled))
    if not by_year:
        return None
    summary_label = f"{label}, всего"
    rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
      <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}" id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
        <input type="hidden" name="start_year" value="{{ start_year }}">
        <input type="hidden" name="end_year" value="{{ end_year }}">

        <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type" value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
        <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>

        <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type" value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
        <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>

        <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type" value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
        <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">Россия</label>

        <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type" value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
        <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>

        <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type" value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
        <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>

        {% if sorted_synchronous_area_ids %}
          {% for sa_id in sorted_synchronous_area_ids %}
            {% set sa_name = synchronous_area_names[sa_id] %}
            {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
            {% set sa_value = 'sync_area_' + sa_id|string %}
            {% set is_checked = sa_value in aggregation_types %}
            <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}" name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
            <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}" style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
          {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
305      should_show_totals = {
306          "energy_system_types": {},
307          "union_energy_systems": False,
308          "synchronous_areas": {},
309          "federal_districts": False,
310          "total": False,
311      }
312      if "russia" in agg_types:
313          should_show_totals["total"] = True
314      if "fo" in agg_types:
315          should_show_totals["federal_districts"] = True
316      if "oes" in agg_types:
317          should_show_totals["union_energy_systems"] = True
318      for est_id in sorted_energy_system_type_ids:
319          name_u = (energy_system_type_names.get(est_id) or "").upper()
320          if "ees" in agg_types and "ЕЭС" in name_u:
321              should_show_totals["energy_system_types"][est_id] = True
322          if "tites" in agg_types and "ТИТЭС" in name_u:
323              should_show_totals["energy_system_types"][est_id] = True
324      for sa_id in sorted_synchronous_area_ids:
325          if f"sync_area_{sa_id}" in agg_types:
326              should_show_totals["synchronous_areas"][sa_id] = True
```

8.3. ЗВ: агрегированные суммы с котельными

Адрес: /ecology/totals_summary/zv-vse/

Функциональные возможности и действия пользователя

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются массы ЗВ и топливно-энергетическая база; производные и удельные показатели пересчитываются для агрегата. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

Взаимодействие и источники данных

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

Элементы реализации

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
11  class ZVParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.
13
14      Первоначально копируется из ZVReportGisTek; далее может редактироваться
15      отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
@ecology_bp.route("/totals_summary/<sheet_slug>")
@ecology_bp.route("/totals_summary/<sheet_slug>/")
@login_required
def totals_summary(sheet_slug: str):
    context = _ecology_totals_page_context(sheet_slug)
    if context is None:
        abort(404)
    log_page_visit(
        "Итоговые суммы",
        tab=(context.get("sheet") or {}).get("page_title"),
        details=context_lines(
            ("лист", sheet_slug),
            ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
            ("округление", context.get("rounding_digits")),
        ),
    )
    return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
def build_region_summary_block(
    groups: list[dict[str, Any]],
    label: str,
    *,
    source_columns: list[dict[str, Any]],
    visible_columns: list[dict[str, Any]],
    digits: int,
    key_prefix: str,
    emission_digits: int | None = None,
    specific_digits: int | None = None,
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> dict[str, Any] | None:
    """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
    by_year: dict[int, list[list[Any]]] = defaultdict(list)
    for group in groups:
        for year, filled in (group.get("filled_by_year") or {}).items():
            if year is None or not filled:
                continue
            by_year[int(year)].append(list(filled))
    if not by_year:
        return None
    summary_label = f"{label}, всего"
    rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник:
app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
  91        <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}"
id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
  92          <input type="hidden" name="start_year" value="{{ start_year }}">
  93          <input type="hidden" name="end_year" value="{{ end_year }}">
  94
  95          <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type"
value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
  96          <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>
  97
  98          <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type"
value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
  99          <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>
 100
 101          <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type"
value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
 102          <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-
primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width:
80px;">Россия</label>
 103
 104          <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type"
value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
 105          <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>
 106
 107          <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type"
value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
 108          <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>
 109
 110          {% if sorted_synchronous_area_ids %}
 111            {% for sa_id in sorted_synchronous_area_ids %}
 112              {% set sa_name = synchronous_area_names[sa_id] %}
 113              {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
 114              {% set sa_value = 'sync_area_' + sa_id|string %}
 115              {% set is_checked = sa_value in aggregation_types %}
 116              <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}"
name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
 117              <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}"
style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
 118            {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
 305      should_show_totals = {
 306          "energy_system_types": {},
 307          "union_energy_systems": False,
 308          "synchronous_areas": {},
 309          "federal_districts": False,
 310          "total": False,
 311      }
 312      if "russia" in agg_types:
 313          should_show_totals["total"] = True
 314      if "fo" in agg_types:
 315          should_show_totals["federal_districts"] = True
 316      if "oes" in agg_types:
 317          should_show_totals["union_energy_systems"] = True
 318      for est_id in sorted_energy_system_type_ids:
 319          name_u = (energy_system_type_names.get(est_id) or "").upper()
 320          if "ees" in agg_types and "ЕЭС" in name_u:
 321              should_show_totals["energy_system_types"][est_id] = True
 322          if "tites" in agg_types and "ТИТЭС" in name_u:
 323              should_show_totals["energy_system_types"][est_id] = True
 324      for sa_id in sorted_synchronous_area_ids:
 325          if f"sync_area_{sa_id}" in agg_types:
 326              should_show_totals["synchronous_areas"][sa_id] = True
```

8.4. 3В: агрегированные суммы без котельных

Адрес: /ecology/totals_summary/zv-bez-kot/

Функциональные возможности и действия пользователя

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются массы ЗВ и топливно-энергетическая база; производные и удельные показатели пересчитываются для агрегата. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

Взаимодействие и источники данных

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

Элементы реализации

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
11  class ZVParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.
13
14      Первоначально копируется из ZVReportGisTek; далее может редактироваться
15      отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
64  @ecology_bp.route("/totals_summary/<sheet_slug>")
65  @ecology_bp.route("/totals_summary/<sheet_slug>/")
66  @login_required
67  def totals_summary(sheet_slug: str):
68      context = _ecology_totals_page_context(sheet_slug)
69      if context is None:
70          abort(404)
71      log_page_visit(
72          "Итоговые суммы",
73          tab=(context.get("sheet") or {}).get("page_title"),
74          details=context_lines(
75              ("лист", sheet_slug),
76              ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
77              ("округление", context.get("rounding_digits")),
78          ),
79      )
80      return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
3112  def build_region_summary_block(
3113      groups: list[dict[str, Any]],
3114      label: str,
3115      *,
3116      source_columns: list[dict[str, Any]],
3117      visible_columns: list[dict[str, Any]],
3118      digits: int,
3119      key_prefix: str,
3120      emission_digits: int | None = None,
3121      specific_digits: int | None = None,
3122      norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
3123  ) -> dict[str, Any] | None:
3124      """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
3125      by_year: dict[int, list[list[Any]]] = defaultdict(list)
3126      for group in groups:
3127          for year, filled in (group.get("filled_by_year") or {}).items():
3128              if year is None or not filled:
3129                  continue
3130              by_year[int(year)].append(list(filled))
3131      if not by_year:
3132          return None
3133      summary_label = f"{label}, всего"
3134      rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
  91        <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}"
id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
  92          <input type="hidden" name="start_year" value="{{ start_year }}">
  93          <input type="hidden" name="end_year" value="{{ end_year }}">
  94
  95          <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type"
value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
  96          <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>
  97
  98          <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type"
value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
  99          <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>
 100
 101          <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type"
value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
 102          <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-
primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width:
80px;">Россия</label>
 103
 104          <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type"
value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
 105          <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>
 106
 107          <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type"
value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
 108          <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>
 109
 110          {% if sorted_synchronous_area_ids %}
 111            {% for sa_id in sorted_synchronous_area_ids %}
 112              {% set sa_name = synchronous_area_names[sa_id] %}
 113              {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
 114              {% set sa_value = 'sync_area_' + sa_id|string %}
 115              {% set is_checked = sa_value in aggregation_types %}
 116              <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}"
name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
 117              <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}"
style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
 118            {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
    should_show_totals = {
        "energy_system_types": {},
        "union_energy_systems": False,
        "synchronous_areas": {},
        "federal_districts": False,
        "total": False,
    }
    if "russia" in agg_types:
        should_show_totals["total"] = True
    if "fo" in agg_types:
        should_show_totals["federal_districts"] = True
    if "oes" in agg_types:
        should_show_totals["union_energy_systems"] = True
    for est_id in sorted_energy_system_type_ids:
        name_u = (energy_system_type_names.get(est_id) or "").upper()
        if "ees" in agg_types and "ЕЭС" in name_u:
            should_show_totals["energy_system_types"][est_id] = True
        if "tites" in agg_types and "ТИТЭС" in name_u:
            should_show_totals["energy_system_types"][est_id] = True
    for sa_id in sorted_synchronous_area_ids:
        if f"sync_area_{sa_id}" in agg_types:
            should_show_totals["synchronous_areas"][sa_id] = True
```

9. Тексты подсказок формул

9.1. Редактор текстов подсказок формул

Адрес: /ecology/formulas/

Функциональные возможности и действия пользователя

Таблица содержит страницу применения, уровень агрегации, ячейку или показатель и текст формулы с пояснением. Есть предпросмотр, раскрываемая область редактирования, фильтры и поиск. Сохранение отправляет formula_key и formula_text в отдельный POST-маршрут; сброс удаляет переопределение и возвращает исходный текст. Доступ проверяется на сервере и для страницы, и для сохраняющих операций. Изменяется только содержание подсказок «i»: формулы вычисления, алгоритмы импорта и числовые данные не переписываются.

Взаимодействие и источники данных

Определения и исходные тексты поступают из operating_tes_formula_registry. Эффективный текст выбирается между сохраненным переопределением и исходным текстом; общий FormulaTextOverrideStore выполняет хранение и кеширование. Подсказки используются в отчетах, анализе и расчетах ТЭС. Журнал фиксирует прежний и новый текст.

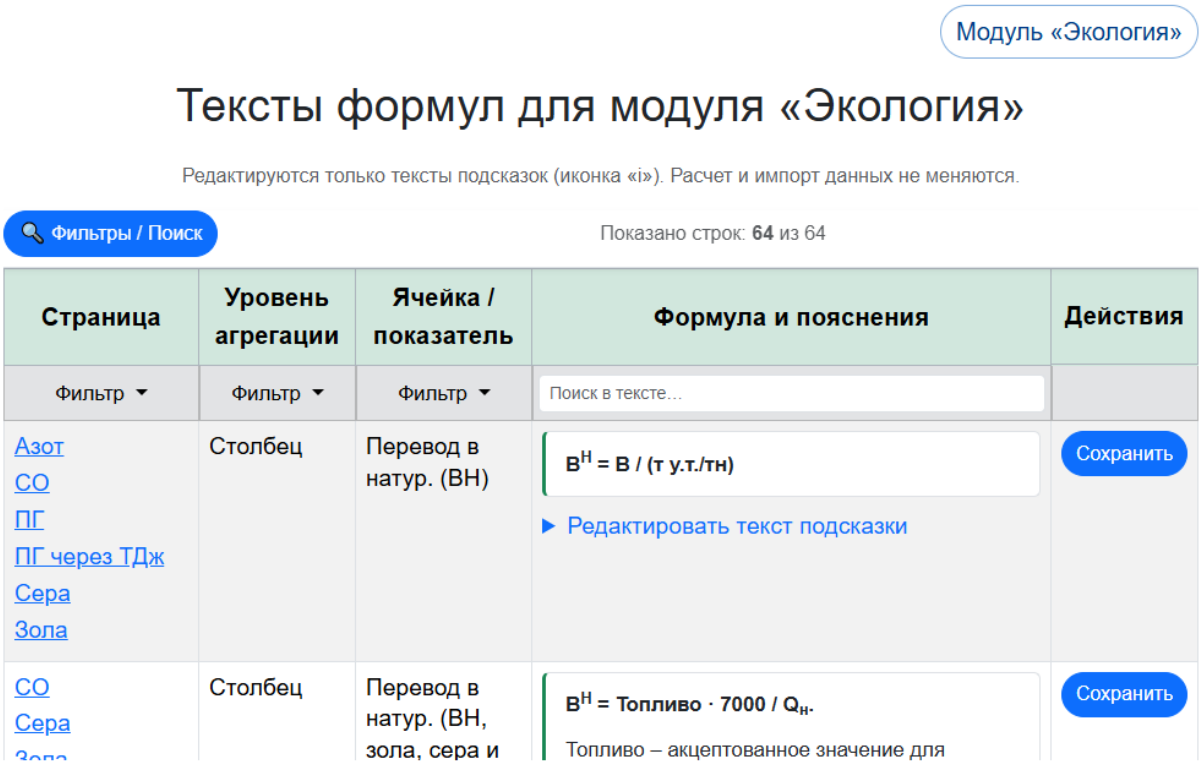


Рисунок 12. Редактор текстов подсказок формул без сохранения изменений. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Элементы реализации

Модель OperatingTesFormulaText. Источник: app/ecology/models/formula_text/operating_tes_formula_text_model.py, строки 11–25.

```
class OperatingTesFormulaText(db.Model, AuditMixin):
    """Только текст формулы; расчет значений ячеек не меняется."""

    __tablename__ = "gs_eco_operating_tes_formula_texts"
    __table_args__ = (
        db.UniqueConstraint("formula_key", name="uq_gs_eco_operating_tes_formula_texts_key"),
        {"schema": SCHEMA_ECOLOGY},
    )

    # Уникальный идентификатор записи
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Ключ формулы (иконка «i» на страницах расчета)
    formula_key = db.Column(db.String(128), nullable=False, index=True)
    # Текст формулы (подсказка; на расчет ячеек не влияет)
    formula_text = db.Column(db.Text, nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 152–163.

```
@ecology_bp.route("/formulas/")
@login_required
def operating_tes_formulas():
    if not can_edit_ecology(current_user):
        flash("Недостаточно прав для редактирования текстов формул.", "danger")
        return redirect(url_for("ecology_bp.hub"))
    log_page_visit("Тексты формул")
    return render_template(
        "ecology/operating_tes/operating_tes_formulas.html",
        page_title="Тексты формул для модуля «Экология»",
        formula_rows=list_formulas_for_admin(),
    )
```

Сервис и расчетная логика. Источник: app/ecology/services/formula_text/operating_tes_formula_text_services.py, строки 35–43.

```
def ot_formula_text(formula_key: str, default: str | None = None) -> str:
    """Эффективный текст формулы: переопределение из БД или значение по умолчанию."""
    definition = get_formula_def(formula_key)
    fallback = default if default is not None else (definition.default_text if definition else "")
    overrides = get_formula_text_overrides()
    override = overrides.get(formula_key)
    if override is not None and str(override).strip() != "":
        return str(override).strip()
    return str(fallback).strip()
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_formulas.html, строки 203–228.

```
                    {% for row in formula_rows %}
                    <tr data-formula-key="{{ row.key }}"
                        data-filter-page-lines="{{ row.page_lines|join('||')|e }}"
                        data-filter-aggregation="{{ row.aggregation_level }}"
                        data-filter-cell="{{ row.cell_name }}">
                        <td class="small ec-formula-page-cell">
                            <ul class="list-unstyled mb-0">
                                {% for page_line in row.page_lines %}
                                <li class="mb-1">
                                    {% if ' (/' in page_line %}
                                    {% set page_parts = page_line.rsplit(' (', 1) %}
                                    <a href="{{ page_parts[1][:-1] }}">{{ page_parts[0] }}</a>
                                    {% else %}{{ page_line }}{% endif %}
                                </li>
                                {% endfor %}
                            </ul>
                        </td>
                        <td class="small">{{ row.aggregation_level }}</td>
                        <td class="small">{{ row.cell_name }}</td>
                        <td>
                            <div class="ecology-formula-preview" role="region" aria-
label="Предпросмотр: {{ row.cell_name }}" aria-live="polite">{{ row.formula_text }}</div>
                            <details class="ecology-formula-editor">
                                <summary>Редактировать текст подсказки</summary>
                                <textarea class="form-control form-control-sm ec-formula-text-input
fuel-param-input" rows="4"
                                          data-default="{{ row.default_text|e }}"
                                          aria-label="Текст формулы {{ row.cell_name }}">{{
row.formula_text }}</textarea>
```

Сохранение подсказки. Источник: app/ecology/routes/ecology_routes.py, строки 166–184.

```
@ecology_bp.route("/formulas/save", methods=["POST"])
@login_required
def operating_tes_formulas_save():
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    data = request.get_json(silent=True) or {}
    key = str(data.get("formula_key") or "")
    new_text = str(data.get("formula_text") or "")
    log_context = context_lines(("формула", key))
    try:
        previous_text = ot_formula_text(key)
        save_formula_text_override(
            formula_key=key,
            formula_text=new_text,
        )
        db.session.commit()
    except ValueError as exc:
        db.session.rollback()
        return jsonify(ok=False, error=str(exc)), 400
```

Формирование списка определений. Источник: app/ecology/services/formula_text/operating_tes_formula_text_services.py, строки 93–109.

```
def list_formulas_for_admin(*, page: str | None = None) -> list[dict[str, Any]]:
    overrides = get_formula_text_overrides()
    out: list[dict[str, Any]] = []
    for item in iter_formula_defs(page=page):
        out.append(
            {
                "key": item.key,
                "page_labels": item.page_labels(),
                "page_lines": list(item.page_label_lines()),
                "aggregation_level": item.aggregation_level,
                "cell_name": item.cell_name,
                "default_text": item.default_text,
                "formula_text": ot_formula_text(item.key),
                "is_customized": item.key in overrides,
            }
        )
    return out
```

10. Рассмотрение данных, переданных из модуля «Топливо»

10.1. Рассмотрение пакета данных из модуля «Топливо»

Адрес: /ecology/operating_tes/fuel_submissions/<int:submission_id>/

Функциональные возможности и действия пользователя

Страница сравнивает принятые показатели и переданный пакет за конкретный год и категорию. Для каждого отличия выводятся показатель, территория, единица, прежнее значение, новое значение и разница. Фильтры показателя и территории работают в таблице. При статусе pending и наличии прав доступны «Акцептовать» и «Не принимать»; ссылка «Назад без решения» ничего не утверждает. Неизвестный пакет дает 404, недостаточные права на решение дают 403, повторное рассмотрение или замена новой отправкой возвращает конфликт 409.

Взаимодействие и источники данных

Пакет передается из «Топлива» как отдельный JSON-снимок. До акцепта он не заменяет принятую топливную базу расчетов. Принятие заменяет только снимок выбранных версии, года и категории, снимает прежние ручные переопределения действующих ТЭС и переводит пакет в accepted. Отклонение сохраняет ранее принятую базу и задает rejected. Блокировка снимка сериализует отправку и рассмотрение. Категории operating, new_lt100 и new_ge100 не объединяются в одну запись.

Элементы реализации

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 125–143.

```
125  @ecology_bp.route("/operating_tes/fuel_submissions/<int:submission_id>/", methods=["GET", "POST"])
126  @login_required
127  def operating_tes_fuel_review(submission_id):
128      from app.ecology.services.fuel_exchange_services import (
129          get_submission, review_fuel_submission, submission_differences,
130      )
131      from app.ecology.services.operating_tes_page_services import format_cell
132
133      submission = get_submission(submission_id)
134      if submission is None:
135          abort(404)
136      from app.ecology.services.fuel_summary_constants import FUEL_SUMMARY_LABELS
137
138      from app.ecology.services.fuel_summary_constants import NEW_TES_SCOPE_CAPACITY
139
140      scope_label = FUEL_SUMMARY_LABELS[submission.scope]
141      capacity_class = NEW_TES_SCOPE_CAPACITY.get(submission.scope)
142      return_endpoint = "ecology_bp.operating_tes_table" if submission.scope == "operating" else
"ecology_bp.new_tes_table"
143      can_review = can_edit_ecology(current_user)
```

Сервис и расчетная логика. Источник: app/ecology/services/fuel_exchange_services.py, строки 95–115.

```
  95  def review_fuel_submission(submission_id, accept):
  96      submission = get_submission(submission_id)
  97      if submission is None:
  98          raise ValueError("Пакет не найден в текущей версии базы данных")
  99      snapshot = EcologyFuelSnapshot.query.filter_by(
 100          database_version_id=submission.database_version_id, year_number=submission.year_number,
scope=submission.scope,
 101      ).with_for_update().one()
 102      db.session.refresh(submission)
 103      if submission.status != "pending":
 104          raise ValueError("Пакет уже рассмотрен или заменен новой отправкой. Обновите страницу.")
 105      if accept:
 106          snapshot.payload = submission.payload
 107          # Прежние ручные переопределения не должны перекрывать принятый пакет.
 108          if submission.scope == "operating":
 109              OperatingTesInputValue.query.filter_by(
 110                  database_version_id=submission.database_version_id,
year_number=submission.year_number,
 111              ).delete(synchronize_session=False)
 112      submission.status = "accepted" if accept else "rejected"
 113      submission.reviewed_at = func.now()
 114      db.session.commit()
 115      return submission
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/fuel_review.html, строки 17–48.

```
  17    <div class="table-responsive">
  18      <table id="fuel-review-table" class="table table-bordered table-striped">
  19        <thead class="table-light"><tr>
  20          {% for label in ['Показатель', 'Территория'] %}
  21          <th scope="col">{{ label }}
  22            <button type="button" class="btn btn-sm btn-outline-secondary d-block mt-1 fuel-review-
filter"
  23                    data-column="{{ loop.index0 }}" data-label="{{ label }}"
  24                    aria-label="Фильтр: {{ label }}" aria-haspopup="dialog" aria-expanded="false"
hidden>
  25              <i class="bi bi-funnel" aria-hidden="true"></i> <span>Все</span>
  26            </button>
  27          </th>
  28          {% endfor %}
  29          <th scope="col">Ед. изм.</th><th scope="col">Принято сейчас</th><th scope="col">Новые
данные</th><th scope="col">Разница</th>
  30        </tr></thead>
  31        <tbody>{% for row in differences %}
  32          <tr><th>{{ row.label }}</th><td>{{ row.territory }}</td><td>{{ row.unit }}</td>
  33            <td>{{ format_cell(row.before, digits=0) }}</td><td class="table-warning">{{
format_cell(row.after, digits=0) }}</td><td>{{ format_cell(row.delta, digits=0) }}</td></tr>
  34        {% endfor %}
  35          <tr id="fuel-review-empty" hidden><td colspan="6" class="text-center text-muted">Нет строк,
соответствующих фильтрам.</td></tr>
  36        </tbody>
  37      </table>
  38    </div>
  39    <p id="fuel-review-filter-status" class="text-muted" role="status" hidden></p>
  40    <div class="d-flex gap-2 mb-4">
  41      {% if can_review and submission.status == 'pending' %}
  42      <form method="POST" class="d-flex gap-2">
  43        <input type="hidden" name="csrf_token" value="{% if csrf_token is string %}{{ csrf_token }}{%
else %}{{ csrf_token() }}{% endif %}">
  44        <button name="decision" value="accept" class="btn btn-success btn-lg">Акцептовать</button>
  45        <button name="decision" value="reject" class="btn btn-outline-danger btn-lg">Не
принимать</button>
  46      </form>
  47      {% endif %}
  48      <a class="btn btn-secondary" href="{{ url_for(return_endpoint, territory_slug='vsego',
sheet_slug='toplivo', start_year=request.args.get('start_year', submission.year_number),
end_year=request.args.get('end_year', submission.year_number), view_year=submission.year_number,
rounding_digits=request.args.get('rounding_digits', 1), capacity_class=capacity_class|default(none))
}}">Назад без решения</a>
```

11. Сквозная логика и взаимодействие с другими модулями

11.1. Отчет, выверенные параметры и агрегаты

Последовательность: пользователь выбирает версию БД и год; уполномоченный специалист загружает Excel на странице отчета; импорт разрешает код группы оборудования и год, создает или обновляет исходные строки и возвращает счетчики. Для анализа создаются только отсутствующие параметры, после чего специалист может выверять их отдельно. Лист анализа соединяет эти параметры с топливными и генерационными показателями, вычисляет разности и удельные значения. Агрегированная форма суммирует базовые величины и заново вычисляет производные показатели.

Загрузка ПГ дополнительно инициирует заполнение отсутствующих параметров анализа ПГ. Для ЗВ заполнение предусмотрено при построении аналитических строк. Импорт не является безусловной заменой всех выверенных параметров: существующие строки анализа сохраняются. Пропуск неизвестного года или кода станции и отсутствие найденной группы учитываются отдельными счетчиками; отсутствие группы не следует смешивать с отсутствием кода.

Импорт: обязательные колонки, текущая версия и индекс существующих записей. Источник: app/ecology/services/gis_tek_import_services.py, строки 304–332.

```
304  def import_gis_tek_from_xlsx_bytes(file_bytes: bytes, kind: str, user: str) -> dict:
305      if kind not in REPORT_MODELS:
306          raise ValueError("Неизвестный вид отчетных данных.")
307      logger = _get_logger()
308      t0 = time.perf_counter()
309      df = read_gis_tek_sheet(file_bytes, kind)
310      if "station_numb" not in df.columns or "year_number" not in df.columns:
311          raise ValueError(
312              "В листе нет обязательных столбцов «№_станции» и «Год»."
313          )
314
315      model = REPORT_MODELS[kind]
316      value_attrs = [
317          attr
318          for attr, _label in VALUE_COLUMNS[kind]
319          if kind != REPORT_ZV or attr not in ZV_DERIVED_ATTRS
320      ]
321      version_id = get_current_db_version_id()
322      groups_by_numb = _load_equipment_groups_by_numb(version_id)
323      known_years = _known_year_numbers(version_id)
324
325      existing_query = model.query
326      if version_id is not None:
327          existing_query = existing_query.filter(model.database_version_id == version_id)
328      else:
329          existing_query = existing_query.filter(model.database_version_id.is_(None))
330      existing = {
331          (row.station_numb, row.year_number): row for row in existing_query.all()
332      }
```

Первичное заполнение анализа ПГ после импорта отчета. Источник: app/ecology/services/gis_tek_import_services.py, строки 435–445.

```
435      seeded_params = 0
436      if kind == REPORT_PG:
437          from app.ecology.services.analysis_pg_param_services import (
438              seed_missing_analysis_pg_params_from_reports,
439          )
440          from app.ecology.services.gis_tes_fuel_comparison_constants import (
441              PG_ANALYSIS_SHEET_SLUGS,
442          )
443
444          for sheet_slug in sorted(PG_ANALYSIS_SHEET_SLUGS):
445              seeded_params += seed_missing_analysis_pg_params_from_reports(sheet_slug)
```

Отсутствующие параметры создаются без перезаписи существующих. Источник: app/ecology/services/analysis_param_services.py, строки 64–88.

```
64          existing = {
65              (row.station_numb, row.year_number)
66              for row in param_q.with_entities(
67                  params_model.station_numb, params_model.year_number
68              ).all()
69          }
70          created = 0
71          for report in report_q.all():
72              key = (report.station_numb, report.year_number)
73              if key in existing:
74                  continue
75              db.session.add(
76                  params_model(
77                      sheet_slug=slug,
78                      station_numb=report.station_numb,
79                      year_number=report.year_number,
80                      database_version_id=version_id,
81                      **values_payload(report, fields),
82                  )
83              )
84              existing.add(key)
85              created += 1
86          if created:
87              db.session.commit()
88          return created
```

Отдельное хранилище выверенных параметров. Источник: app/ecology/services/analysis_param_services.py, строки 100–127.

```
100  def upsert_param_values(
101      params_model: Any,
102      reports_model: Any,
103      *,
104      sheet_slug: str,
105      sheet_slugs: frozenset[str],
106      fields: tuple[str, ...],
107      label: str,
108      station_numb: int,
109      year_number: int,
110      values: dict[str, Any],
111  ) -> Any:
112      """Создает или обновляет параметры страницы анализа."""
113      from app.common.services.database_version_filter import get_current_db_version_id
114      from app.extensions import db
115
116      slug = str(sheet_slug or "").strip()
117      if slug not in sheet_slugs:
118          raise ValueError(f"Неизвестный sheet_slug для параметров {label}: {sheet_slug!r}")
119      version_id = get_current_db_version_id()
120      query = params_model.query.filter_by(
121          sheet_slug=slug,
122          station_numb=int(station_numb),
123          year_number=int(year_number),
124      )
125      if version_id is not None:
126          query = query.filter(params_model.database_version_id == version_id)
127      row = query.first()
```

11.2. Передача топлива и расчет выбросов ТЭС

Последовательность: в «Топливе» собирается сводка за год и категорию; отправка создает EcologyFuelSubmission со статусом pending, не меняя содержимое уже принятого снимка. Предыдущий ожидающий пакет того же разреза получает superseded. Специалист «Экологии» сравнивает значения и принимает либо отклоняет пакет. Только accepted заменяет payload снимка выбранного разреза. Расчетные сервисы читают принятые EcologyFuelSnapshot, применяют физические характеристики, коэффициенты очистки и нормативы, затем сводят действующие и обе категории новых ТЭС.

Объекты групп оборудования, их топливные параметры и дополнительные виды топлива принадлежат «Топливу». «Генерация» предоставляет связанные станции, мощность, выработку и сведения для проверки соответствия. «Справочники» задают годы, виды топлива, территориальную и энергетическую иерархию. Выбранная версия БД ограничивает предметный разрез. Журнал отражает изменения и решения, но не является источником расчетных значений.

Отправка пакета без замены принятой базы. Источник: app/ecology/services/fuel_exchange_services.py, строки 62–88.

```
  62  def submit_fuel_year(year, scope="operating"):
  63      if scope not in FUEL_SUMMARY_LABELS:
  64          raise ValueError("Неизвестный раздел сводки")
  65      version_id = _version_id()
  66      from app.ecology.services.fuel_summary_services import load_fuel_summary
  67
  68      inputs = load_fuel_summary([year], scope)
  69      # Один замок на раздел и год сериализует отправку и рассмотрение пакета.
  70      snapshot = EcologyFuelSnapshot.query.filter_by(
  71          database_version_id=version_id, year_number=year, scope=scope,
  72      ).with_for_update().first()
  73      if snapshot is None:
  74          db.session.execute(
  75              insert(EcologyFuelSnapshot).values(
  76                  database_version_id=version_id, year_number=year, scope=scope, payload={},
  77              ).on_conflict_do_nothing(index_elements=["database_version_id", "year_number",
"scope"])
  78          )
  79          snapshot = EcologyFuelSnapshot.query.filter_by(
  80              database_version_id=version_id, year_number=year, scope=scope,
  81          ).with_for_update().one()
  82      payload = encode_inputs(inputs.get(year, {}))
  83      for previous in pending_submissions([year], scope=scope):
  84          previous.status = "superseded"
  85      submission = EcologyFuelSubmission(database_version_id=version_id, year_number=year,
scope=scope, payload=payload, status="pending")
  86      db.session.add(submission)
  87      db.session.commit()
  88      return submission
```

Акцепт или отклонение пакета. Источник: app/ecology/services/fuel_exchange_services.py, строки 95–115.

```
  95  def review_fuel_submission(submission_id, accept):
  96      submission = get_submission(submission_id)
  97      if submission is None:
  98          raise ValueError("Пакет не найден в текущей версии базы данных")
  99      snapshot = EcologyFuelSnapshot.query.filter_by(
 100          database_version_id=submission.database_version_id, year_number=submission.year_number,
scope=submission.scope,
 101      ).with_for_update().one()
 102      db.session.refresh(submission)
 103      if submission.status != "pending":
 104          raise ValueError("Пакет уже рассмотрен или заменен новой отправкой. Обновите страницу.")
 105      if accept:
 106          snapshot.payload = submission.payload
 107          # Прежние ручные переопределения не должны перекрывать принятый пакет.
 108          if submission.scope == "operating":
 109              OperatingTesInputValue.query.filter_by(
 110                  database_version_id=submission.database_version_id,
year_number=submission.year_number,
 111              ).delete(synchronize_session=False)
 112      submission.status = "accepted" if accept else "rejected"
 113      submission.reviewed_at = func.now()
 114      db.session.commit()
 115      return submission
```

Вход передачи из модуля «Топливо». Источник: app/fuel/routes/ecology_summary_routes.py, строки 12–35.

```
@fuel_bp.route("/ecology_summary/", methods=["GET", "POST"])
@login_required
def ecology_summary():
    scope = request.args.get("scope", "operating")
    if scope not in FUEL_SUMMARY_LABELS:
        abort(404)
    context = build_operating_tes_table_context(
        "vsego", "toplivo", start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"), rounding_digits=request.args.get("rounding_digits"),
        view_year=request.args.get("view_year"), fuel_source=True, fuel_scope=scope,
    )
    can_send = bool(current_user.has_admin)
    if request.method == "POST":
        if not can_send:
            return "Недостаточно прав", 403
        year = context["view_year"]
        if year not in context["range_years"]:
            return "Год не входит в выбранный диапазон", 400
        try:
            submit_fuel_year(year, scope=scope)
        except FuelSummaryError as exc:
            return str(exc), 409
        flash(f'{FUEL_SUMMARY_LABELS[scope]}: данные за {year} год переданы в модуль «Экология» и
ожидают акцепта.', "success")
        return redirect(url_for("fuel_bp.ecology_summary", start_year=context["start_year"],
end_year=context["end_year"], rounding_digits=context["rounding_digits"], view_year=year, scope=scope if
scope != "operating" else None))
```

11.3. Формулы, единицы измерения и отсутствующие данные

Для действующих ТЭС перевод в натуральное топливо для золы, серы и CO использует B × 7000 / Qн. Прямой расчет CO2 использует B × KCO2; энергетический вариант дополнительно использует ТДж и т CO2/ТДж. Улавливание рассчитывается как Мотх × (1 − коэффициент). В новых ТЭС масса ЗВ = расход в тыс. т у.т. × норматив в кг/т у.т. дает тонны, а масса CO2 = расход × норматив в т/т у.т. дает тыс. т. Перед суммированием выполняется согласование единиц.

Для удельных показателей важны единицы знаменателя. Расчетные ТЭС используют приведенную энергию (E + Q × 1,163) / 1000 в млрд кВт·ч; анализ отчетов использует свою функцию приведения W_en + Teplo / 859,8 × 1000 в млн кВт·ч. Эти выражения описывают текущую реализацию разных форм, а не замену одной формулы другой. Отсутствующее значение и нулевой знаменатель обрабатываются явно; знак отсутствия данных не является автоматически измеренным нулем. Документ не подтверждает научную аттестацию методики или полноту исходных данных.

Натуральный расход топлива по теплоте сгорания. Источник: app/ecology/services/operating_tes_calc_services.py, строки 35–41.

```
def to_natural_by_heat(b_tce: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Перевод для золы, серы и CO: BH = B · 7000 / Qн (ккал/кг)."""
    b = _d(b_tce)
    qh = _d(chars.get("qh_kcal")) if chars else None
    if b is None or qh is None or qh == 0:
        return None
    return b * Decimal("7000") / qh
```

Учет улавливания или подавления. Источник: app/ecology/services/operating_tes_calc_services.py, строки 124–130.

```
def apply_capture(motkh: Any, eta: Any) -> Decimal | None:
    """Мвыб = Мотх·(1 − η)."""
    m = _d(motkh)
    if m is None:
        return None
    e = _d(eta) or Decimal("0")
    return m * (Decimal("1") - e)
```

Приведенная энергия расчетных ТЭС. Источник: app/ecology/services/operating_tes_calc_services.py, строки 133–143.

```
133  def equivalent_energy_billion_kwh(e_mln_kwh: Any, q_ths_gcal: Any) -> Decimal | None:
134      """Приведенная энергия, млрд кВт·ч = (E + Q·1,163) / 1000."""
135      e = _d(e_mln_kwh)
136      q = _d(q_ths_gcal)
137      if e is None and q is None:
138          return None
139      e = e or Decimal("0")
140      q = q or Decimal("0")
141      from app.ecology.services.operating_tes_constants import GCAL_TO_MWH
142
143      return (e + q * GCAL_TO_MWH) / Decimal("1000")
```

Приведенная энергия анализа отчетных данных. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 1990–2000.

```
1990  def _brought_energy_mln_kwh(w_en: Any, teplo: Any) -> Any:
1991      """Приведенная энергия, млн кВт·ч: W_en + Teplo / 859,8 · 1000."""
1992      energy = _clean_number(w_en)
1993      heat = _clean_number(teplo)
1994      if energy is None and heat is None:
1995          return None
1996      with localcontext() as ctx:
1997          ctx.prec = SIGNIFICANT_DIGITS
1998          ctx.rounding = ROUND_HALF_UP
1999          total = (energy or Decimal(0)) + (heat or Decimal(0)) / Decimal(str(KCAL_PER_KWH)) * 1000
2000          return float(+total)
```

Защита от отсутствующей или нулевой базы деления. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 1963–1972.

```
1963  def _safe_divide(numerator: Any, denominator: Any) -> Any:
1964      """Безопасное деление: None при пустых значениях или нулевом знаменателе."""
1965      left = _clean_number(numerator)
1966      right = _clean_number(denominator)
1967      if left is None or right is None or right == 0:
1968          return None
1969      with localcontext() as ctx:
1970          ctx.prec = SIGNIFICANT_DIGITS
1971          ctx.rounding = ROUND_HALF_UP
1972          return float(+(left / right))
```

Единицы массы в расчетах новых ТЭС. Источник: app/ecology/services/new_tes_calc_services.py, строки 12–35.

```
 12  def zv_mass_tonnes(b_tce: Any, u_kg_per_tce: Any) -> Decimal | None:
 13      """М, т = В, тыс. т у.т. × У, кг/т у.т."""
 14      b = _d(b_tce)
 15      u = _d(u_kg_per_tce)
 16      if b is None or u is None:
 17          return None
 18      return b * u
 19
 20
 21  def zv_mass_thousand_tonnes(b_tce: Any, u_kg_per_tce: Any) -> Decimal | None:
 22      """М, тыс. т = (В, тыс. т у.т. × У, кг/т у.т.) / 1000."""
 23      tonnes = zv_mass_tonnes(b_tce, u_kg_per_tce)
 24      if tonnes is None:
 25          return None
 26      return tonnes / Decimal("1000")
 27
 28
 29  def pg_mass_thousand_tonnes(b_tce: Any, u_t_per_tce: Any) -> Decimal | None:
 30      """М CO2, тыс. т = В, тыс. т у.т. × У, т/т у.т."""
 31      b = _d(b_tce)
 32      u = _d(u_t_per_tce)
 33      if b is None or u is None:
 34          return None
 35      return b * u
```

11.4. Обмен Excel и интерактивные действия

Отчеты ПГ/ЗВ имеют собственные маршруты импорта и выгрузки. Анализ экспортируется без ограничения текущей страницей (per_page=all). Выгрузка

итоговых сумм учитывает выбранные уровни и период. Для действующих ТЭС и сводного расчета Excel формируется по вкладкам и годовым блокам. В текущем new_tes_routes.py отдельного маршрута выгрузки новых ТЭС нет; при описании этой страницы нельзя автоматически приписывать ей экспорт действующих ТЭС.

Ввод в разрешенную числовую ячейку передает PATCH с годом, полем и ключами строки. Сервер проверяет пользователя, допустимый лист, вид поля и корректность числа; листы только для просмотра возвращают отказ. Формы решения по пакету и сохранения подсказок используют POST. Тексты подсказок не исполняются как программные формулы и не меняют расчетный алгоритм.

Excel: расчетные вкладки и годовые блоки. Источник: app/ecology/services/tes_excel_export_services.py, строки 23–31.

```
23  def build_tes_excel_stream(territory_slug, args, *, summary=False):
24      builder = build_calculation_summary_context if summary else build_operating_tes_table_context
25      tabs = CALCULATION_SUMMARY_TABS if summary else CALC_SHEETS
26      years = get_year_columns(args.get('start_year'), args.get('end_year'))
27      filters = {key: args.get(key) for key in ('rounding_digits', 'coefficient_rounding_digits')}
28      if not summary:
29          filters['affiliation'] = args.get('affiliation', 'all')
30      workbook = Workbook()
31      workbook.remove(workbook.active)
```

HTML/JavaScript: PATCH разрешенной числовой ячейки. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 1068–1089.

```
1068            const tr = el.closest('tr');
1069            const cell = el.closest('td');
1070            clearSaveState(el);
1071            state.saving = true;
1072            try {
1073              const response = await fetch(el.dataset.updateUrl || updateUrl, {
1074                method: 'PATCH',
1075                credentials: 'same-origin',
1076                headers: {
1077                  'Content-Type': 'application/json',
1078                  'X-Requested-With': 'XMLHttpRequest',
1079                  'X-CSRFToken': CSRF || ''
1080                },
1081                body: JSON.stringify({
1082                  year: el.dataset.year,
1083                  territory: el.dataset.territory,
1084                  affiliation: el.dataset.affiliation || table.dataset.affiliation,
1085                  row_key: tr?.dataset.rowKey,
1086                  field: el.dataset.field || tr?.dataset.editableField,
1087                  value: value
1088                })
1089              });
```

HTML/JavaScript: сохранение параметра анализа. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 1098–1118.

```
1098            if (input.value === (input.dataset.original || '')) return;
1099            const tr = input.closest('tr');
1100            const td = input.closest('td');
1101            recalcDerivedRow(tr);
1102            input.disabled = true;
1103            if (td) td.classList.remove('is-save-success', 'is-save-error');
1104            try {
1105              const response = await fetch(updateUrl, {
1106                method: 'PATCH',
1107                credentials: 'same-origin',
1108                headers: {'Content-Type': 'application/json', 'X-CSRFToken': CSRF || ''},
1109                body: JSON.stringify({
1110                  year: input.dataset.year,
1111                  numb: input.dataset.numb,
1112                  field: input.dataset.field,
1113                  value: input.value,
1114                  rounding_digits: roundingDigits,
1115                  emission_rounding_digits: emissionRoundingDigits,
1116                  specific_rounding_digits: specificRoundingDigits,
1117                  norm_zv: {{ norm_zv_selected|default([])|tojson }}
1118                })
```

Проверки сохранения норматива новых ТЭС. Источник: app/ecology/routes/new_tes_routes.py, строки 101–126.

```
@ecology_bp.route(
    "/new_tes/<territory_slug>/<sheet_slug>/value/",
    methods=["PATCH"],
)
@login_required
def new_tes_update_value(territory_slug: str, sheet_slug: str):
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    if get_territory(territory_slug) is None or get_calc_sheet(sheet_slug) is None:
        abort(404)
    capacity = request.args.get("capacity_class", CAPACITY_LE100_KEY)
    if capacity not in CAPACITY_CLASSES:
        abort(404)
    data = request.get_json(silent=True) or {}
    year = data.get("year")
    field = str(data.get("field") or "").strip()
    row_key = str(data.get("row_key") or "").strip()
    try:
        year_int = int(year)
    except (TypeError, ValueError):
        return jsonify(ok=False, error="Некорректный год"), 400
    raw = data.get("value")
    try:
        amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
    except Exception:
        return jsonify(ok=False, error="Некорректное число"), 400
```

Сброс только текста подсказки. Источник: app/ecology/routes/ecology_routes.py, строки 196–216.

```
@ecology_bp.route("/formulas/reset", methods=["POST"])
@login_required
def operating_tes_formulas_reset():
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    data = request.get_json(silent=True) or {}
    key = str(data.get("formula_key") or "").strip()
    if not key:
        return jsonify(ok=False, error="Не указан ключ формулы."), 400
    previous_text = ot_formula_text(key)
    reset_formula_text_override(key)
    db.session.commit()
    log_cell_change(
        "Тексты формул",
        f"Текст формулы {key}",
        previous_text,
        ot_formula_text(key),
        context=context_lines(("формула", key), ("действие", "сброс к исходному тексту")),
        number=False,
    )
    return jsonify(ok=True)
```

12. Полнота описания модуля «Экология» и источники

12.1. Перечень экранных представлений

Раздел	Представление	Адрес или пример
3.1	Главная страница модуля	/ecology/
3.2	Парниковые газы: отчетные данные ГИС ТЭС	/ecology/gis_tek/pg/
3.3	Загрязняющие вещества: отчетные данные ГИС ТЭС	/ecology/gis_tek/pollutants/
4.1	Парниковые газы: анализ с котельными	/ecology/analysis/pg-vse/
4.2	Парниковые газы: анализ без котельных	/ecology/analysis/pg-bez-kot/
4.3	Загрязняющие вещества: анализ с котельными	/ecology/analysis/zv-vse/
4.4	Загрязняющие вещества: анализ без котельных	/ecology/analysis/zv-bez-kot/
5.1	Данные по модулю Топливо	/ecology/operating_tes/vsego/toplivo/
5.2	Характеристики топлив и продуктов сгорания	/ecology/operating_tes/vsego/khar-topl/
5.3	Коэффициенты очистки и подавления	/ecology/operating_tes/vsego/koeff/
5.4	Зола: выбросы твердых веществ	/ecology/operating_tes/vsego/zola/
5.5	Сера: выбросы диоксида серы	/ecology/operating_tes/vsego/sera/
5.6	Азот: выбросы оксидов азота	/ecology/operating_tes/vsego/azot/
5.7	CO: выбросы монооксида углерода	/ecology/operating_tes/vsego/co/
5.8	ПГ: прямой расчет CO2	/ecology/operating_tes/vsego/pg/
5.9	ПГ через ТДж: расчет CO2 по энергии топлива	/ecology/operating_tes/vsego/pg-tj/
5.10	Объемы и удельные выбросы действующих ТЭС	/ecology/operating_tes/vsego/obemy/
6.1	Принятые топливные показатели новых ТЭС	/ecology/new_tes/vsego/toplivo/
6.2	Нормативы удельных выбросов новых ТЭС	/ecology/new_tes/vsego/spravochnik/
6.3	Объемы выбросов новых ТЭС	/ecology/new_tes/vsego/obemy/
7.1	Общие топливные показатели действующих и новых ТЭС	/ecology/calculation_summary/vsego/toplivo/
7.2	Объемы выбросов по России	/ecology/calculation_summary/vsego/russia/
7.3	Удельные выбросы по России	/ecology/calculation_summary/vsego/specific/

Раздел	Представление	Адрес или пример
7.4	Динамика изменения выбросов	/ecology/calculation_summary/vsego/dynamics/
8.1	ПГ: агрегированные суммы с котельными	/ecology/totals_summary/pg-vse/
8.2	ПГ: агрегированные суммы без котельных	/ecology/totals_summary/pg-bez-kot/
8.3	ЗВ: агрегированные суммы с котельными	/ecology/totals_summary/zv-vse/
8.4	ЗВ: агрегированные суммы без котельных	/ecology/totals_summary/zv-bez-kot/
9.1	Редактор текстов подсказок формул	/ecology/formulas/
10.1	Рассмотрение пакета данных из модуля «Топливо»	/ecology/operating_tes/fuel_submissions/<int:submission_id>/

12.2. Скрытые совместимые вкладки сводного расчета

12.2.1. Действующие ТЭС

Адрес: /ecology/calculation_summary/vsego/operating/. Показывает расчет действующих ТЭС без добавления новых. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 7. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

12.2.2. Новые ТЭС: обе категории

Адрес: /ecology/calculation_summary/vsego/new/. Суммирует объемы новых ТЭС le100 и gt100 без действующих. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 7. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

12.2.3. Новые ТЭС le100

Адрес: /ecology/calculation_summary/vsego/new-le100/. Показывает отдельную категорию новых ТЭС, сопоставленную с new_lt100. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 7. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

12.2.4. Новые ТЭС gt100

Адрес: /ecology/calculation_summary/vsego/new-gt100/. Показывает отдельную категорию новых ТЭС, сопоставленную с new_ge100. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 7. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

Основные и скрытые совместимые вкладки. Источник: app/ecology/services/calculation_summary_services.py, строки 16–29.

```
16  CALCULATION_SUMMARY_TABS = (
17      {"slug": "toplivo", "sheet_name": "Данные по модулю Топливо"},
18      {"slug": "russia", "sheet_name": "Объемы выбросов по России"},
19      {"slug": "specific", "sheet_name": "Удельные выбросы по России"},
20      {"slug": "dynamics", "sheet_name": "Динамика изменения выбросов"},
21  )
22
23  # Скрытые вкладки остаются доступны по ранее сохраненным ссылкам.
24  _LEGACY_TABS = (
25      {"slug": "operating", "sheet_name": "Объемы выбросов для действующих ТЭС"},
26      {"slug": "new", "sheet_name": "Объемы выбросов для новых ТЭС"},
27      {"slug": "new-le100", "sheet_name": "Объемы выбросов для новых ТЭС до 100 МВт"},
28      {"slug": "new-gt100", "sheet_name": "Объемы выбросов для новых ТЭС 100 МВт и более"},
29  )
```

Выбор источников совместимых сводных представлений. Источник: app/ecology/services/calculation_summary_services.py, строки 162–196.

```
162  def build_calculation_summary_context(territory_slug, sheet_slug, **filters):
163      tab = next((tab for tab in (*CALCULATION_SUMMARY_TABS, *_LEGACY_TABS) if tab["slug"] ==
sheet_slug), None)
164      if tab is None:
165          return None
166      source_sheet = "toplivo" if sheet_slug == "toplivo" else "obemy"
167      new_filters = {key: value for key, value in filters.items() if key !=
"coefficient_rounding_digits"}
168
169      def new_context(capacity):
170          return build_new_tes_table_context(
171              territory_slug, source_sheet, capacity_class=capacity, **new_filters,
172          )
173
174      if sheet_slug == "dynamics":
175          context = _build_emission_dynamics(territory_slug, filters)
176      elif sheet_slug in {"new-le100", "new-gt100"}:
177          context = new_context(sheet_slug.removeprefix("new-"))
178          if context is None:
179              return None
180      elif sheet_slug == "operating":
181          context = build_operating_tes_table_context(territory_slug, source_sheet, **filters)
182      else:
183          le100, gt100 = new_context("le100"), new_context("gt100")
184          if le100 is None or gt100 is None:
185              return None
186          if sheet_slug == "toplivo":
187              context = build_operating_tes_table_context(territory_slug, source_sheet, **filters)
188              context["rows"] = _sum_rows(context, [context, le100, gt100])
189              context["page_title"] = "Топливные показатели действующих и новых ТЭС (модуль Топливо)"
190              context["page_title_html"] = context["page_title"]
191          else:
192              new_sources = [_transpose_new_volumes(source) for source in (le100, gt100)]
193              context = {**new_sources[0], "rows": _sum_rows(new_sources[0], new_sources)}
194              if sheet_slug in {"russia", "specific"}:
195                  operating = build_operating_tes_table_context(territory_slug, source_sheet,
**filters)
196                  context = {**operating, "rows": _sum_rows(operating, [operating, context])}
```

12.3. Маршруты страниц и служебные операции

Метод	Адрес	Обработчик
GET	/ecology/calculation_summary/	calculation_summary_index
GET	/ecology/calculation_summary/<territory_slug>/export.xlsx	calculation_summary_export
GET	/ecology/calculation_summary/<territory_slug>/<sheet_slug>/	calculation_summary
GET	/ecology/	hub
GET	/ecology/formulas/	operating_tes_formulas
POST	/ecology/formulas/save	operating_tes_formulas_save
POST	/ecology/formulas/reset	operating_tes_formulas_reset
GET	/ecology/gis_tek/pg/	gis_tek_pg
POST	/ecology/gis_tek/pg/import	gis_tek_pg_import
GET	/ecology/gis_tek/pg/export.xlsx	gis_tek_pg_export
GET	/ecology/gis_tek/pollutants/	gis_tek_pollutants
POST	/ecology/gis_tek/pollutants/import	gis_tek_pollutants_import
GET	/ecology/gis_tek/pollutants/export.xlsx	gis_tek_pollutants_export
GET	/ecology/totals_summary	totals_summary_index
GET	/ecology/totals_summary/	totals_summary_index
GET	/ecology/totals_summary/<sheet_slug>	totals_summary
GET	/ecology/totals_summary/<sheet_slug>/	totals_summary
GET	/ecology/totals_summary_export	totals_summary_export_index
GET	/ecology/totals_summary_export/	totals_summary_export_index
GET	/ecology/totals_summary/<sheet_slug>/export	totals_summary_export
GET	/ecology/totals_summary/<sheet_slug>/export/	totals_summary_export
GET	/ecology/analysis/<sheet_slug>/	gis_tes_fuel_comparison
GET	/ecology/analysis/<sheet_slug>/export.xlsx	gis_tes_fuel_comparison_export
PATCH	/ecology/analysis/<sheet_slug>/value/	gis_tes_fuel_comparison_update_value
GET	/ecology/new_tes/	new_tes_hub
GET	/ecology/new_tes/<territory_slug>/	new_tes_territory
GET	/ecology/new_tes/<territory_slug>/<sheet_slug>/	new_tes_table
PATCH	/ecology/new_tes/<territory_slug>/<sheet_slug>/value/	new_tes_update_value
GET	/ecology/operating_tes/	operating_tes_hub
GET	/ecology/operating_tes/<territory_slug>/	operating_tes_territory
GET	/ecology/operating_tes/<territory_slug>/<sheet_slug>/	operating_tes_table
GET	/ecology/operating_tes/<territory_slug>/export.xlsx	operating_tes_export
GET, POST	/ecology/operating_tes/fuel_submissions/<int:submission_id>/	operating_tes_fuel_review

Метод	Адрес	Обработчик
PATCH	/ecology/operating_tes/<territory_slug>/<sheet_slug>/value/	operating_tes_update_value

Маршруты верхнего уровня operating_tes/, new_tes/, calculation_summary/ и totals_summary/ перенаправляют к исходному листу. Export, value, import, save и reset являются операциями существующих страниц, а не дополнительными экранными формами. Варианты totals_summary со слешем и без него ведут к одному представлению. Технические HEAD и OPTIONS Flask в число отдельных прикладных операций не включены.

12.4. Проверка и ограничения

Описание сверено с маршрутами, моделями, сервисами, HTML и включаемыми JavaScript текущей рабочей копии. Снимки подтверждают внешний вид только просмотренных страниц и конкретного состояния БД на дату получения. Изменяющие операции и научная корректность расчетов не испытывались на рабочем сервере. При подаче заявки заявитель по-прежнему подтверждает официальное название программы, авторов, правообладателя, объем и дату завершения. Описание модуля не заполняет эти сведения автоматически.

Сведения по предоставленному шаблону регистрации приведены в разделе 1, описание модуля «Экология» в разделах 2–12. Описание «Справочников» оформлено отдельным файлом. Стили основного текста, заголовков, таблиц и иллюстраций унаследованы из ранее примененного отчета «Отчет_Этап 1_АРМ_ГС_2026-НК.docx». Моноширинный SourceCode сохранен для листингов. Оглавление включает только разделы настоящего документа и обновляется средствами Word.